

Schedule-Free On-Demand Water Transport in Bergen

September 30, 2026

Abstract

This report documents the design of the real-time dispatch logic for a small-boat, on-demand water transport service in Bergen, Norway. The service does not exist yet: rather than optimizing an existing system, the project builds a model of how such a service would operate, replacing fixed timetables with a dispatch policy that recomputes itself every few minutes. The pieces covered are, in order: a navigable routing module that computes real, coastline-respecting travel times between the four candidate stops; a synthetic demand model anchored in official Norwegian population and employment data; a simulation environment, implemented to the Gymnasium interface standard, that hosts the dispatch problem; four fixed, rule-based dispatch heuristics, built one on top of the previous, each verified by hand-built controlled cases and then compared against the others across seven fleet sizes; a reinforcement-learning agent trained with proximal policy optimization to learn a dispatch policy directly, including a search over twelve different reward formulas that determined how the agent is trained and a discussion of a software bug found and fixed during that search; and a final, full-scale comparison of the trained agent against the strongest of the four heuristics, on the same twelve-boat, full-day instance and the same held-out evaluation seeds used throughout. At that full scale, the trained agent does not outperform the heuristics, and this report says so plainly rather than only reporting the smaller-scale result where it did.

Contents

1	Introduction and Motivation	3
2	Related Work	3
3	Navigable Routing	4
4	Demand Model	5
4.1	Catchment areas and masses	5
4.2	From population and employment to origin-destination intensity	6
4.3	From intensity to Poisson arrival rates	7
4.4	Arrivals and group sizes	8
4.5	Persons vs. groups	10
5	Simulation Environment	10
5.1	State	10
5.2	Actions	10
5.3	Transition	11
5.4	Reward	11

6	Dispatch Heuristics: H0 to H3	11
6.1	H0: the nearest-available rule	11
6.2	Coordinating several free boats at once	12
6.3	Verification at three scales	13
6.4	Known limitations of H0	16
6.5	A second coordination gap: claims that outlive a single step	16
6.6	H1: adding a reservation that survives the trip	16
6.7	H2: a cost function, but only where H0 already looks	17
6.8	H3: one cost, no separate tiers	18
6.9	How the four rules compare	18
7	Fleet-Size Sensitivity	19
8	Reinforcement Learning Agent	20
8.1	Method	20
8.2	Implementation, observation, and action	20
8.3	Reward function	21
8.4	Reward-shaping experiments	22
8.5	A fuller search: eight reward formulas at full training budget	23
8.6	Fresh demand on every reset	26
8.7	Training run	26
8.8	Scaling up: training at the full instance	28
9	Results: Agent vs. Reference Policy at Small Scale	29
9.1	Aggregate comparison, five seeds	29
9.2	Detailed comparison on a single seed	30
10	Results: Full-Day Comparison	31
10.1	Aggregate result, five seeds	32
10.2	Reading the result honestly	33
11	Assumptions and Limitations	33
12	Conclusion	34
A	Where Everything Lives in This Project	35

1 Introduction and Motivation

Bergen is a coastal city whose public transport network is structured as a hub-and-spoke system centered on the city center, a layout imposed by the surrounding fjords and mountains. As the city grows and tourism increases, this network is under increasing pressure, and land-based expansion is constrained by the same geography that makes it congested in the first place. The fjords that divide the city are, at the same time, an underused transport corridor.

This project explores a small-boat, on-demand water transport service as a complementary mode. Two principles guide its design. First, the service is not modeled as a profit-making venture (public transport of this kind is rarely self-financing, and its justification is environmental and related to congestion relief, not revenue); the objective is to deliver a good level of service efficiently, not to maximize income. Second, the flexible layer of the service has no fixed timetable: a passenger requests a trip and a boat is dispatched to serve it, subject to a waiting-time guarantee, and the dispatch policy is recomputed at short, regular intervals rather than following a pre-published schedule. Section 2 situates this "schedule-free" operating philosophy, and the static water-taxi model it builds on, within the closely related literature.

Scope of this report. It covers the routing and demand modeling that feed the project (Sections 3 and 4), the simulation environment that hosts the dispatch problem (Section 5), four fixed dispatch heuristics built one on top of the previous and verified against that environment (Section 6), how those four heuristics compare once the fleet size is allowed to vary (Section 7), a reinforcement-learning agent trained to learn a dispatch policy directly (Section 8), a first comparison of that agent against the reference policy at a small, two-boat scale (Section 9), and the project's main result, a full-scale comparison of the agent against the strongest heuristic on the twelve-boat, full-day instance the rest of the project is built around (Section 10). The environment is intentionally simple (four stops, direct point-to-point trips, a small set of rules), chosen so that its mechanics could be verified before adding complexity to the model of how boats and passengers behave. Section 11 states this and the project's other simplifying choices explicitly, together, rather than leaving them implicit, and an appendix at the end maps every piece of this report to where it lives in the project's own repository.

2 Related Work

Three papers anchor the modeling choices described in this report.

Gu and Wallace [2021] formulate a static location, fleet-sizing, and routing model for a water-taxi system connecting the same four stops used in this project (Kleppestø, Laksevåg, Bryggen, and Sandviken). It is the closest precedent in terms of the physical instance, and it is used in Section 4 as one reference point for the scale of daily demand.

Braathen et al. [2024] study autonomous ferry scheduling under labor regulations, comparing autonomous and manually operated fleets. The demand-generation approach in Section 4 is adapted from their multiplicative, gravity-style formulation, and the group-size statistics they report are used to calibrate the group-size distribution used here.

Sánchez-Martínez et al. [2016] develop a schedule-free operating paradigm for high-frequency transit control. Their central point, that abandoning fixed schedules in favor of real-time, resource-aware control can improve service quality provided the underlying control problem can be solved effectively, is the conceptual starting point for treating dispatch in this project as a control problem rather than a scheduling problem.

3 Navigable Routing

Travel times between the four stops are computed from a navigable-routing module rather than from straight-line (Haversine) distances. An earlier, straight-line version of the travel-time matrix was found, on visual inspection, to draw lines that cross land near Bryggen, which sits inside the Vågen bay next to the Nordnes peninsula (straight-line distances are therefore not usable directly and are corrected here).

Water/land classification. A basemap tile mosaic covering the four stops is downloaded at zoom level 14, which corresponds to a fixed Web Mercator resolution of $156543.03392/2^{14} \approx 9.55$ m/pixel. Because Web Mercator distorts distances by a factor of $1/\cos(\text{latitude})$, and Bergen sits at approximately 60.4°N , the real ground resolution is $9.55 \times \cos(60.4^\circ) \approx 4.7$ m/pixel. Each pixel is classified as water or land by its Euclidean distance, in RGB space, to a reference water color sampled from the basemap style (threshold of 15 color units); the resulting mask is pixel-for-pixel aligned with the source map, since both are produced by the same Web Mercator projection formula and the same Earth radius.

Graph and shortest paths. Each water pixel becomes a node in an 8-connected graph (4 orthogonal and 4 diagonal neighbors), with edge weights equal to the real distance between pixel centers; land pixels are simply absent from the graph, so no edge can cross land. The four stops are snapped to the nearest water pixel, restricted to the graph’s main connected component, to avoid anchoring a stop to an isolated puddle unconnected to open water. A multi-source Dijkstra search (`scipy.sparse.csgraph.dijkstra`) then computes the shortest navigable path between every pair of stops in a single call, exact because all edge weights are non-negative real distances.

Design speed and resulting matrix. Rather than calibrating vessel speed against an existing ferry line, a fixed design speed of 30 km/h (≈ 16.2 knots) is used for the whole fleet, treated as a design decision for the small on-demand vessels rather than an empirical calibration. Table 1 reports the resulting travel-time matrix, and Table 2 compares it against the straight-line matrix it replaces: routing around land adds between 6% and 24% to travel distance depending on the pair, with Laksevåg (whose corrected location sits further inside the Puddefjorden) showing the largest correction because the detour around the Nordnes peninsula is large relative to its own distance to the other stops.

Table 1: Navigable travel-time matrix (minutes), 30 km/h design speed.

	Kleppestø	Laksevåg	Bryggen	Sandviken
Kleppestø	-	5.3	11.5	9.6
Laksevåg	5.3	-	8.6	8.6
Bryggen	11.5	8.6	-	6.8
Sandviken	9.6	8.6	6.8	-

Table 2: Straight-line vs. navigable travel times, all six stop pairs.

Pair	Straight-line (min)	Navigable (min)	Difference
Laksevåg ↔ Bryggen	7.0	8.6	+23.8%
Kleppestø ↔ Sandviken	8.7	9.6	+10.5%
Bryggen ↔ Sandviken	6.3	6.8	+8.9%
Kleppestø ↔ Laksevåg	4.9	5.3	+7.8%
Kleppestø ↔ Bryggen	10.7	11.5	+7.3%
Laksevåg ↔ Sandviken	8.1	8.6	+6.4%

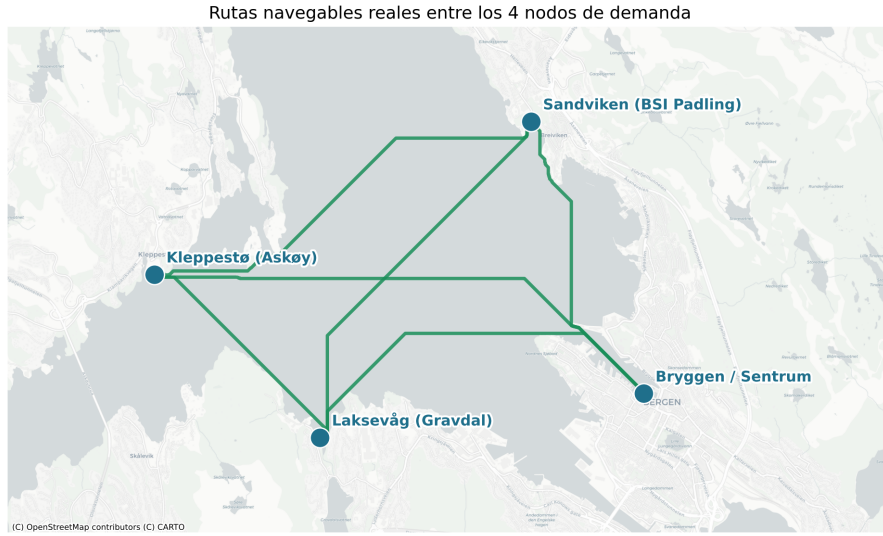


Figure 1: The six navigable routes between the four demand stops, computed by Dijkstra search over the water-classified pixel graph.

The known limitation of this method is that an 8-connected grid only offers eight travel directions per step, so a shortest path can zig-zag slightly instead of following a perfect diagonal when the true direction falls between two of the eight allowed ones; the resulting overcost is a few percent in the worst case and does not affect the central conclusion that the method reliably avoids land.

4 Demand Model

No real demand data exists for this service, since it does not operate yet. Demand is therefore synthesized, but anchored in real open data wherever possible.

4.1 Catchment areas and masses

Population and employment are read from official Statistics Norway (SSB) 250-metre grids. For each of the four stops, a catchment area was hand-drawn by the project team in QGIS; every *grunnkrets* (Norway’s smallest statistical reporting unit) that the drawn area touches, even partially, is included in full for that stop, and duplicate assignments at shared boundaries are resolved by keeping each *grunnkrets* with the stop it overlaps the most. Population and employment are then summed over the SSB grid cells inside the resulting area. Table 3 reports the resulting masses, and Figure 2 shows which *grunnkretser* were assigned to each stop.

Table 3: Catchment area, population, and employment by node (SSB grids, 2025).

Node	Grunnkretser	Area (km ²)	Population	Employment
Kleppestø (Askøy)	28	80.6	25,153	7,550
Laksevåg (Gravdal)	18	14.9	15,986	7,760
Bryggen / Sentrum	67	20.2	27,695	52,213
Sandviken (BSI Padling)	21	14.6	11,109	4,106

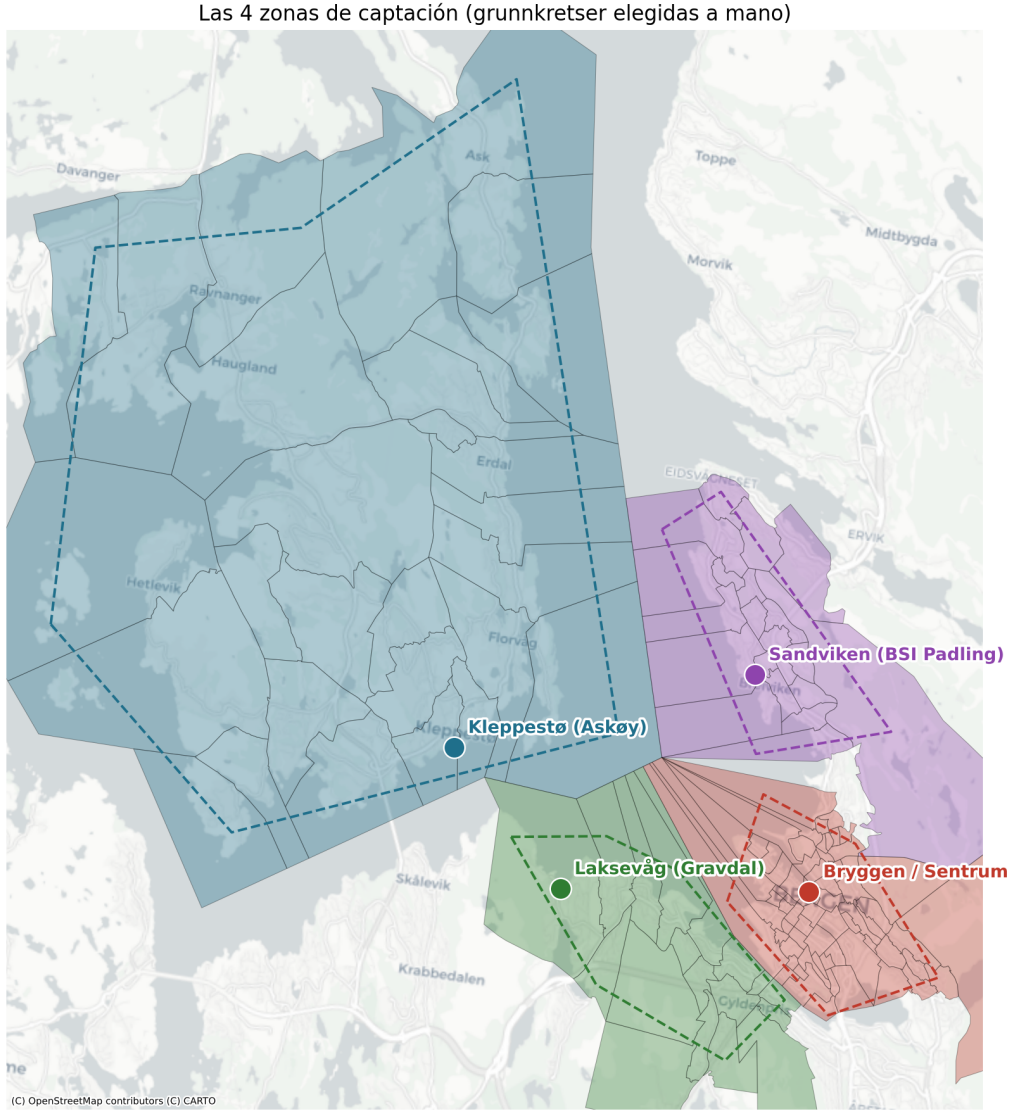


Figure 2: The four hand-drawn catchment areas (dashed outline) and the grunnkretser assigned to each node (filled).

4.2 From population and employment to origin-destination intensity

Braathen et al. [2024] generate synthetic demand using a multiplicative popularity score that combines different factors associated with the attractiveness of stations, travel times, weekdays, and hours. This project adopts the general logic of using a score to represent the relative attractiveness or intensity of demand, but replaces the judgment-based popularity factors with observable population, employment, and university-related measures for the Bergen study area.

The score used in this project is defined as follows.

Let $\text{pop}(n)$ and $\text{emp}(n)$ be the population and employment of node n from Table 3, and let $u(n) \in [0, 1]$ be the university weighting factor described below. Masses are first normalized against the largest node, so that both scales end up comparable before being multiplied:

$$\text{pop}_{\text{norm}}(n) = \frac{\text{pop}(n)}{\max_m \text{pop}(m)}, \quad \text{emp}_{\text{norm}}(n) = \frac{\text{emp}(n)}{\max_m \text{emp}(m)} \cdot u(n). \quad (1)$$

$u(n)$ compensates for the fact that registered employment does not count students, and Bergen is a university city; no open source of fine-grained enrollment data was found, so $u(n)$ is fixed by hand for now (1.0 at Bryggen and Sandviken, reduced at Kleppestø and Laksevåg), declared here as a design choice rather than measured data. A third, "mixed" mass averages the two, used at times of day with no clear commute direction:

$$\text{mix}(n) = \frac{1}{2}(\text{pop}_{\text{norm}}(n) + \text{emp}_{\text{norm}}(n)). \quad (2)$$

Each of the four time bands in Table 4 assigns a role (population, employment, or mixed) to the origin side and to the destination side of every trip; write $w(n, \text{role})$ for whichever of the three masses above that role selects. The origin-destination intensity for band b and the ordered pair (o, d) , $o \neq d$, is then

$$I_b(o, d) = w(o, \text{role}_{\text{origin}}(b)) \cdot w(d, \text{role}_{\text{destination}}(b)) \cdot f_{\text{dist}}(o, d) \cdot v(b), \quad (3)$$

where $v(b)$ is the band's relative volume factor (Table 4) and f_{dist} is a distance-decay term not present in Braathen et al. [2024]'s formulation, added in this project:

$$f_{\text{dist}}(o, d) = \exp\left(-\frac{t(o, d)}{\tau}\right), \quad \tau = 15 \text{ min}, \quad (4)$$

with $t(o, d)$ the navigable travel time from Table 1. $I_b(o, d)$ has no unit of its own; it is a relative weight, only meaningful compared to the other 11 pairs of the same band, and is turned into an actual passenger count in the next step.

Table 4: Time bands, relative demand volume, and origin/destination roles.

Band b	Hours	$v(b)$	Role _{origin} / Role _{destination}
Morning	06-09	1.00	population / employment
Midday	09-15	0.40	mixed / mixed
Afternoon	15-18	1.00	employment / population
Evening	18-24	0.15	mixed / mixed

4.3 From intensity to Poisson arrival rates

Every hour of the day inherits the intensity of the band it belongs to (a step function across bands, not a smoothed hourly curve): $I_h(o, d) := I_{b(h)}(o, d)$ for hour h in band $b(h)$. This is computed for all 24 operating hours and all 12 ordered pairs, covering all four bands, not a subset.

The day's total passenger budget is set as a percentage of the population of the four catchment zones of Table 3 (not the total population of Bergen and Askøy), scaled down on weekends:

$$S = p \cdot P_{\text{zonas}} \cdot f_{\text{semana}}, \quad P_{\text{zonas}} = \sum_n \text{pop}(n), \quad (5)$$

where p is `demanda.porcentaje_poblacion_dia` in the demand model's configuration file (currently 10%, an explicit, changeable team parameter, not a value taken from either reference

paper), $P_{\text{zonas}} = 79,943$ is the sum of the population column of Table 3, and f_{semana} is 1.0 on weekdays and 0.4 on weekends. S is a number of *passengers*; dividing by the mean group size $\bar{g} = 8.6$ (Section 4.4) gives the day’s total number of *groups*, $G = S/\bar{g}$, which is then distributed across all hour-pair cells in proportion to their share of the day’s total intensity:

$$\lambda(o, d, h) = \frac{I_h(o, d)}{\sum_{h', o', d'} I_{h'}(o', d')} \cdot G. \quad (6)$$

Within each hour, the number of arriving groups for pair (o, d) is drawn $N \sim \text{Poisson}(\lambda(o, d, h))$, and each group’s arrival minute is drawn uniformly within that hour.

As a reference point for the order of magnitude of S : Gu and Wallace [2021], who study the same four stops, assume 300 travel requests per day in their case study. That number is for a water-taxi service, a different kind of offering (booked, point-to-point, presumably priced and scheduled differently from a public-transport-style service), so it is not directly comparable to S here; it is mentioned only because it is a verifiable, same-instance data point, not as a validation of the 10% figure.

4.4 Arrivals and group sizes

Passenger arrivals within each hour follow the Poisson process described above. Braathen et al. [2024] report a mean group size of 8.6 passengers with standard deviation 2.6, but do not specify a sampling distribution for it (their own group size is a deterministic function of the popularity score pop_{pg} , not a random draw). Since group size is a count of people, a discrete, non-negative distribution is used instead of a continuous one: a $\text{Binomial}(n = 40, p = 0.215)$, calibrated by matching moments to Braathen et al.’s reported mean and standard deviation, which reproduces the target mean exactly (8.60) and the target standard deviation closely (2.598 against 2.6).

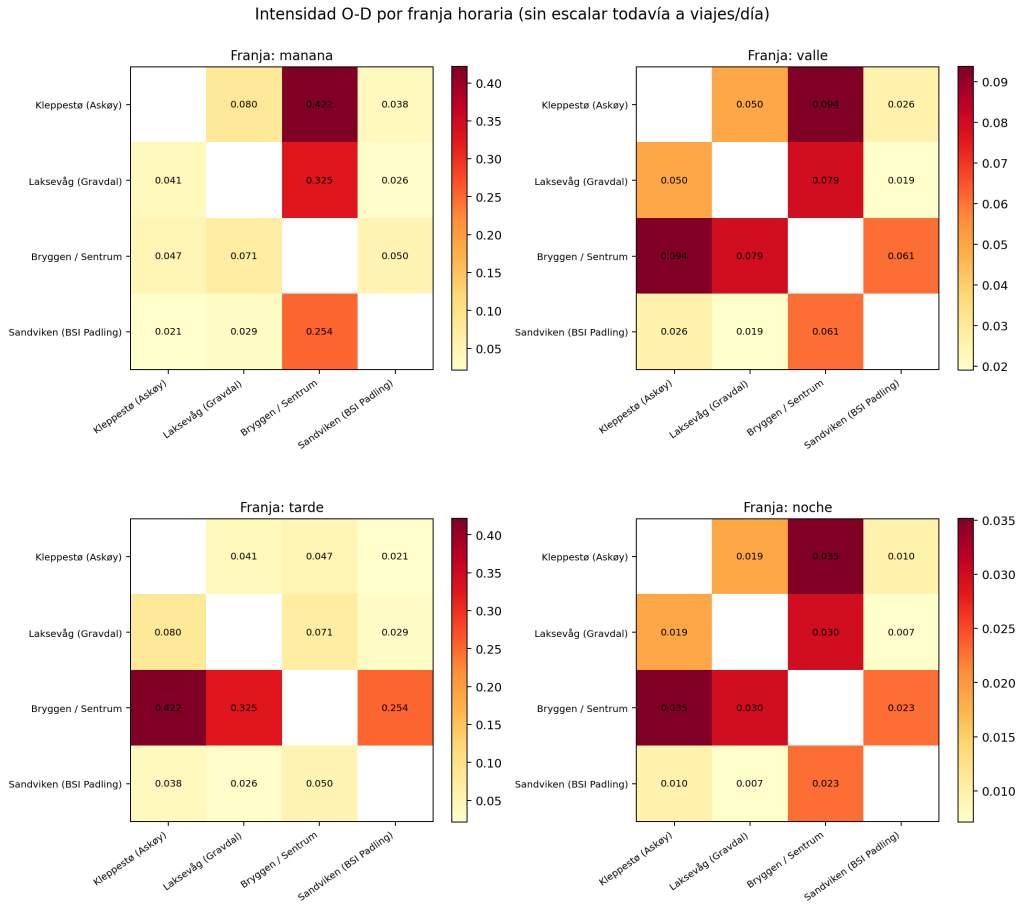


Figure 3: Origin-destination intensity $I_b(o, d)$ by time band. Morning traffic concentrates toward Bryggen; the afternoon band reverses the pattern.

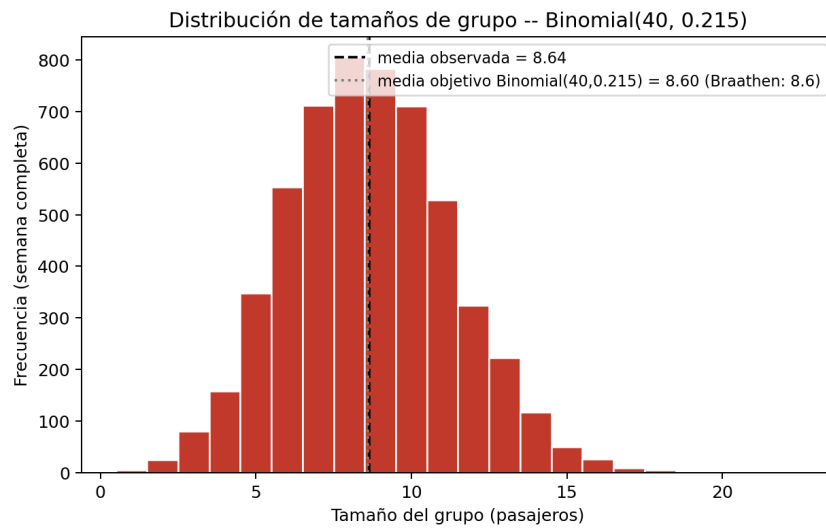


Figure 4: Observed group-size distribution over a simulated week, against the target Binomial(40, 0.215).

4.5 Persons vs. groups

A group, as generated above, is the atomic unit of the demand model. For the dispatch simulation environment (Section 5), each group can optionally be exploded into individual persons, each inheriting the same origin, destination, and arrival time as the group they came from; this changes the atomic unit seen by the simulator’s capacity and boarding logic without altering when or where demand appears.

5 Simulation Environment

This is a first, simple iteration. The environment described in this section establishes a working structure for the dispatch problem, not the final model the project targets. It is deliberately minimal: four stops, direct point-to-point boat trips, and a small set of rules, chosen so that the environment itself (its mechanics, its bookkeeping, its interface) could be verified before adding any complexity to the model of how boats and passengers behave. Extensions are expected as the project continues.

The dispatch problem is modeled as a Markov decision process and implemented as a discrete-time simulation environment following the Gymnasium interface standard (`reset()` / `step()`), which is not a library of pre-built simulators but an interface contract: the environment implements the entire boat-and-passenger logic itself, and only its shape is standardized, so that a reinforcement-learning library can later drive it without modification. The environment advances in fixed 2-minute steps, rather than jumping between discrete events, for simplicity and because it maps directly onto the `step()` call.

5.1 State

The state has three parts. For each boat: origin node, destination node (equal to the origin if idle), minutes remaining until arrival, current occupancy, and whether it is free to receive a new order. For demand: one queue per origin-destination pair (twelve pairs for four nodes, not one queue per node), holding the number of people waiting and the waiting time of the oldest person in that specific pair. For time: minute of day and day of week. This rich, dictionary-shaped state is used for logging and inspection; a separate fixed-size numeric vector is derived from it for the learning agent (Section 8.2 gives its exact composition and size), built from one-hot node encodings for each boat’s origin and destination, a normalized time-to-arrival and occupancy per boat, normalized counts and waiting times per pair queue, and two plain normalized scalars for time (minute of day divided by the length of the day, day of week divided by six) rather than a cyclical encoding: every simulated run is one self-contained episode that starts at a fixed hour and ends within the same day, so the wraparound a cyclical encoding exists to smooth out (23:59 rolling into 00:00, Sunday into Monday) never actually occurs inside a single episode, and a plain linear value carries the same information with two entries instead of four. Two fields present in the dictionary-shaped state are deliberately left out of this numeric vector: whether a boat is free is redundant once its origin and destination one-hot encodings and its time-to-arrival are already present (free is exactly the case origin equals destination and time-to-arrival is zero), and including it as a separate entry would only repeat information already available to the network in a different form.

5.2 Actions

Every 2 minutes, each free boat receives one of five orders: travel to one of the four nodes, or wait. A boat already en route ignores any new order and continues to its destination; it is not redirected mid-trip. Boarding follows a fixed rule, not chosen by the policy: when a free boat at node A is ordered to travel to node B , it boards strictly from the queue for the pair (A, B) ,

in arrival order, up to capacity, and travels directly to B , where everyone aboard disembarks. Trips are therefore always direct, point-to-point journeys between exactly two stops; a boat never carries passengers with mixed destinations, since it can only ever board from the one queue that matches the trip it was just ordered to make. Ordering a free boat to travel to its own current node is treated identically to the explicit wait order: both leave the boat's destination equal to its origin, so neither boards anyone, advances the boat, nor incurs the per-step movement penalty of Section 8.3; the event log used for the metrics reported throughout this document records both forms of staying put under the same label, so that waiting time is never undercounted simply because a policy expressed it as "go to where I already am" rather than the dedicated wait action.

5.3 Transition

Each step executes, in order: (1) apply the chosen action to every free boat, boarding from the matching pair queue as described above; (2) advance all boats in transit by the fixed time step, disembarking everyone aboard any boat that reaches its destination; (3) add to the queues any new arrivals whose arrival time falls within the interval just elapsed (these do not board the boat that just departed in step 1, since they had not yet arrived when boarding happened); (4) compute the step's reward and assemble the new state.

5.4 Reward

The reward returned at every 2-minute step is a single negative scalar that penalizes letting people wait or travel beyond a tolerance and, more lightly, sending a boat under way. This scalar plays no role in the reference dispatch policy of Section 6, which follows a fixed rule and never consults it; it exists purely to give the environment a well-defined objective for a policy that has to learn from experience. Because the reward is meaningful chiefly in that training context, and because its exact shape (in particular, whether the discomfort term is capped, and how strongly movement is penalized) is itself a design choice explored deliberately rather than fixed in advance, its full definition and the reasoning behind each term are given together in Section 8.3, next to the experiments it is meant to explain.

6 Dispatch Heuristics: H0 to H3

Before any policy is asked to learn, it is useful to have a simple, fixed rule that can be verified by hand and used as a point of comparison. This section builds four such rules, one after another. Each one keeps everything the one before it already does and changes exactly one thing, so that any gain (or loss) can be traced back to that one change. H0 is the starting rule: serve local demand first, and only reposition when there is nothing to serve where the boat already is. H1 keeps that same rule but fixes a coordination gap between boats and adds a reservation that survives across simulation steps. H2 adds a cost function, but only inside the choice H1 already makes. H3 goes one step further and removes the local first rule altogether, letting any queue in the system compete on the same footing. The section ends with a side by side comparison of the four rules and the fleet coordination problem that applies to all of them equally.

6.1 H0: the nearest-available rule

The reference policy, referred to here as nearest-available, decides one free boat at a time and follows a single principle: serve the demand waiting at the boat's own current stop before ever considering a trip to reposition toward demand elsewhere. Concretely, when boat b is free at node A , the policy first looks at the twelve pair queues (Section 5.1) restricted to those leaving A ; if any of them holds someone waiting, the boat is sent to the destination whose queue has

been waiting longest, without regard to how many people are in it. Only when no one at all is waiting to leave A does the policy consider repositioning: it looks across every other pair queue in the system and sends the boat, empty, toward the origin node of whichever queue has waited longest, so that the boat can pick that demand up once it arrives.

This rule is deliberately myopic. It never looks two moves ahead: a boat ordered to reposition simply travels to the node with the oldest waiting demand and re-evaluates from scratch once it gets there, rather than planning the pickup and the eventual delivery as a single two-leg trip. It is also strictly local-first, which is a real design choice with a real cost. Consider a boat that becomes free at Bryggen just as five people who arrived a minute earlier ask to go to Laksevåg, while a single person has already been waiting twenty minutes at Kleppestø for a boat to Sandviken. Nearest-available sends the boat to the five people at Bryggen, not to the person who has waited far longer at Kleppestø, because it never compares urgency across nodes while there is still anyone to serve locally. This keeps the rule simple to state and to verify, and it avoids abandoning demand the boat could serve immediately in favor of chasing something that still requires travel time; the cost is that a lightly connected node can, in principle, be left waiting for a long time if the busier nodes never run dry. Because the environment no longer removes anyone from a queue for having waited too long (Section 5.3), this is not a theoretical risk that resolves itself: nothing in the environment stops that wait from growing arbitrarily long if the policy never gets around to it. Whether this actually happens in practice is exactly what the verification below is designed to check.

6.2 Coordinating several free boats at once

A second issue appears only when more than one boat is free at the same node in the same step. Applying the decision rule above independently to each boat, using the same snapshot of the world for both, can make two boats choose the same destination: each one sees, say, eleven people waiting at a shared destination and decides that is the most urgent trip to make, without knowing the other boat just made the identical decision. The first boat to be processed picks up all eleven; the second arrives at the same destination with nobody left to serve, having paid the cost of the trip for nothing. This was not a hypothetical concern. It was found by inspecting the state of a verification run at a single simulated minute and confirmed in the accompanying event log, and it visibly hurt the fraction of demand served in that run.

The fix, applied whenever more than one boat is free in the same step, is to assign boats to actions one at a time rather than all at once from the same snapshot. Boats are processed in a fixed order; after each one is assigned a destination, the pair queue it would draw from is reduced locally, in a scratch copy of the state, by exactly what that boat would pick up, following the same first-in-first-out and capacity rule the environment itself uses when boarding. The next boat in line then applies the decision rule to this already-discounted picture of demand, so a second boat heading to the same busy destination sees only what would genuinely still be waiting for it, not what the first boat already claimed. This coordination step does not change the underlying decision rule; it only prevents two boats from acting on the same stale piece of information. All results reported below use this coordinated version, referred to as *fleet assignment*, never the single-boat rule applied independently to a group. The combination of strict local-first prioritization (Section 6) with this one-at-a-time fleet coordination is what makes nearest-available a competent reference policy rather than a trivial one: it is simple to state, but reasoning correctly about several simultaneously free boats sharing the same demand picture is not automatic, and Section 9 shows it setting a bar that is not trivial for a learned policy to clear.

6.3 Verification at three scales

The coordinated policy was run on three test instances built from the demand model of Section 4, each scaled down from the model’s own 10% daily participation rate so that a run stays easy to follow while still exercising the full mechanics of the environment. Table 5 summarizes the three.

Table 5: The three verification instances: scale, fleet, and purpose.

	Step 1	Step 2	Step 3
Time window	Morning, 06:00 to 09:00	Full day, 06:00 to 24:00	Full week (7 days)
Boats	2	3	3
Groups / people	23 / 202	133 / 1106	620 / 5429
2-minute steps	90	540	7 × up to 540
Purpose	Step-by-step visual check	Aggregate metrics, one day	Weekday/weekend pattern

Step 3 is run as seven independent one-day episodes, one per weekday, rather than as a single continuous week-long clock; the demand generator already applies its own weekday/weekend intensity factor (Table 4 does not show this factor, which multiplies the whole day’s volume by 0.4 on Saturdays and Sundays), so each day already carries the right pattern, and running seven separate days avoids having to decide what an idle boat should do overnight, when no demand exists in the model between hours 0 and 6.

Conservation check. Every person the demand generator creates must be accounted for at the end of a run, either as served, as still waiting in a queue, or as still on board a boat that had not yet reached its destination when the run ended. This identity, generated equals served plus still waiting plus still aboard, was checked automatically on all three runs and held exactly in every case, which is a basic but necessary sanity check given that the environment tracks people individually rather than as aggregate counts.

Table 6: Aggregate results of the reference policy at the three scales.

Metric	Step 1	Step 2	Step 3
Percent served	100.0%	100.0%	99.6%
Mean waiting time (min)	17.4	10.9	10.0
Mean / maximum time in system (min)	27.2 / 54.0	20.5 / 53.6	19.8 / 57.1
Waiting time, 50th / 90th / 95th percentile (min)	15.6 / 33.4 / 33.5	9.4 / 21.6 / 26.3	9.1 / 21.4 / 25.1

Table 6 reports the headline numbers. All three runs serve essentially everyone; the small shortfall in Step 3 (23 people out of 5429, none of them actually lost, just still on board a boat when one of the seven daily windows closed) reflects the mechanical fact that seven independent one-day cutoffs give seven separate chances for a boat to be mid-trip exactly when the clock runs out, not a failure of the policy. Because nothing in the environment discards a slow case anymore (Section 5.3), the percentiles reported here are the real, uncensored distribution of how long a served person actually waited, including its tail, rather than a distribution that quietly excludes whoever would have taken longest.

Step 2, despite carrying 5.5 times the demand of Step 1 with only one additional boat, reports a shorter mean wait and shorter tail percentiles. This is not a contradiction: Step 1 concentrates its entire demand into three peak hours with no slack period for a backlog to clear, while Step 2 spans eighteen hours with alternating busy and quiet bands, giving the fleet repeated opportunities to catch up between peaks.

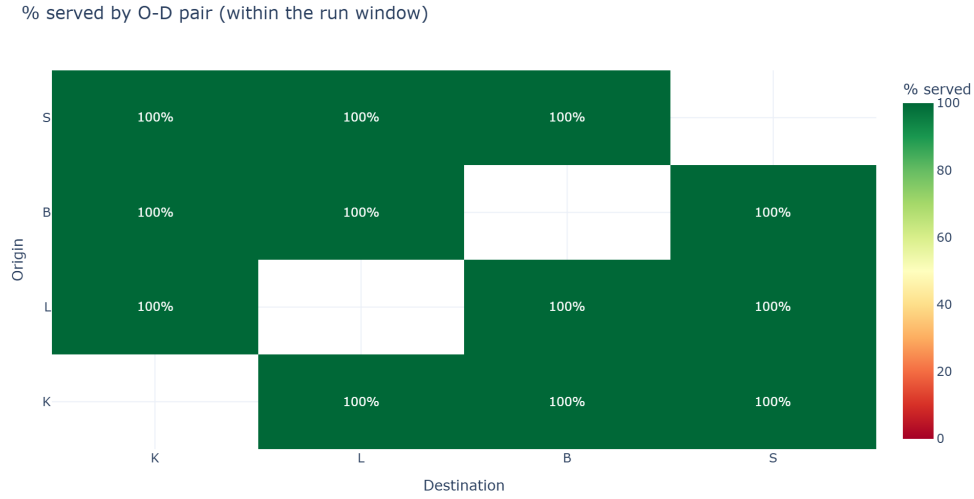


Figure 5: Percent of demand served per origin-destination pair, Step 2 (full day). K, L, B, and S denote Kleppstø, Laksevåg, Bryggen, and Sandviken.

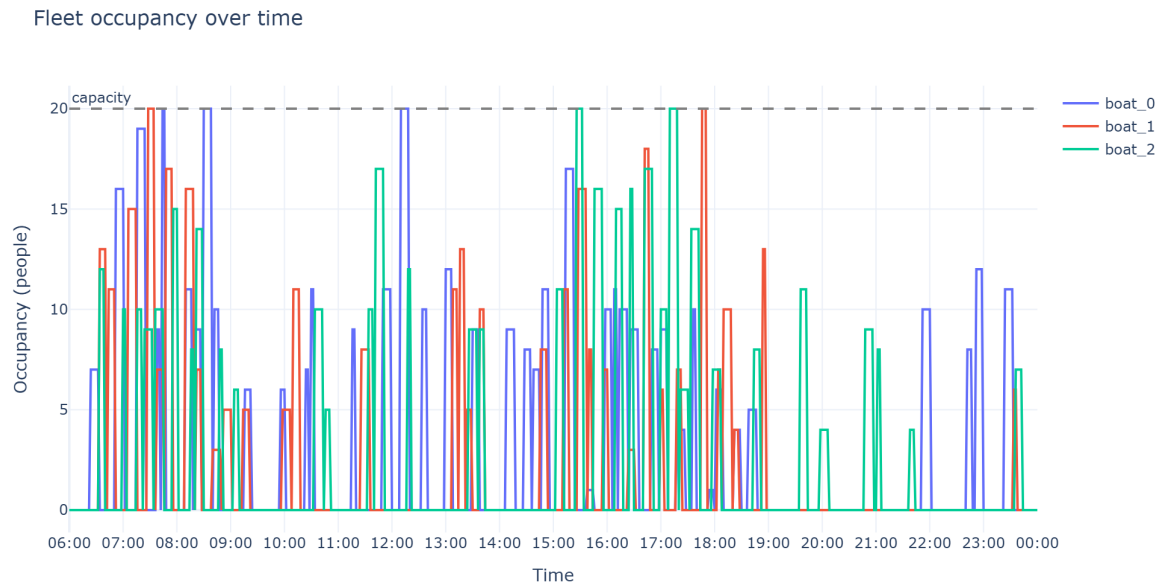


Figure 6: Occupancy of each boat over the full day, Step 2. Both boats spend a large share of the day near full capacity during the morning and afternoon commute bands.

People waiting over time

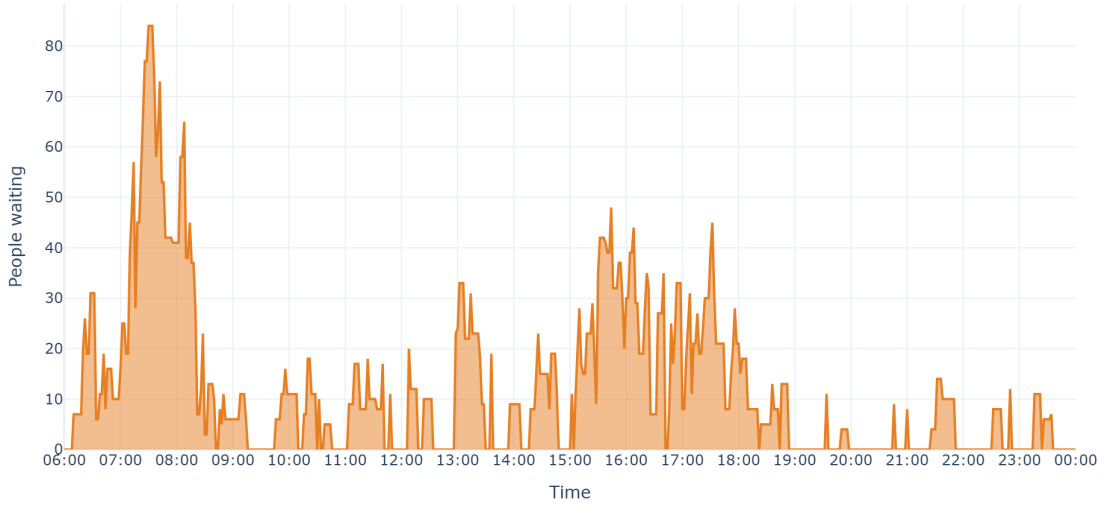


Figure 7: Number of people waiting, summed over all twelve pair queues, over the course of Step 2. The two peaks correspond to the morning and afternoon commute bands of Table 4.

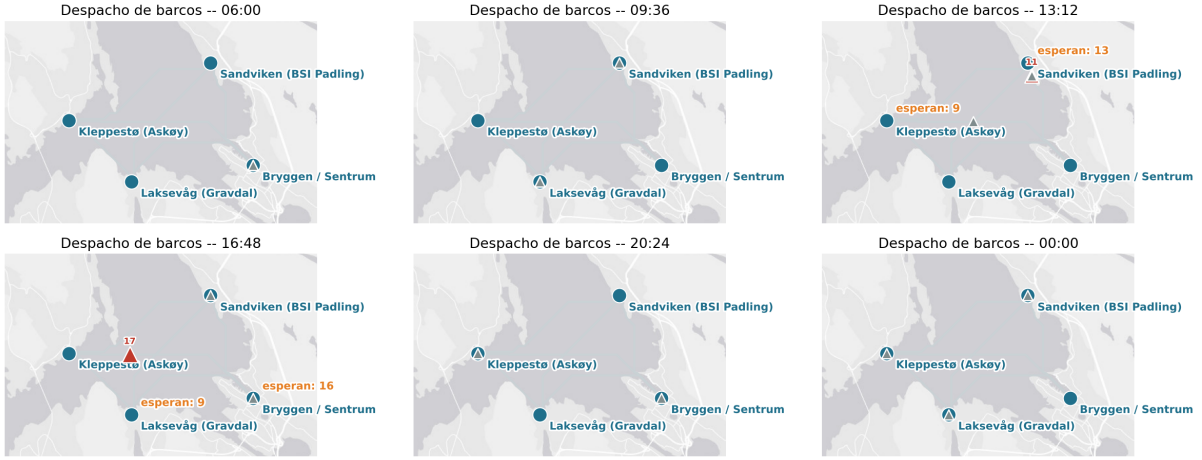


Figure 8: Six snapshots of the fleet over the course of Step 2, taken at even intervals from 06:00 to the following midnight. Orange labels show how many people are waiting at a stop at that instant; a full step-by-step, minute-by-minute view is available as an interactive, browser-based visualization produced by the same code, not reproducible as a static figure here.

Figure 5 breaks the aggregate percent-served figure down by origin-destination pair and confirms that every one of the twelve pairs reaches 100% in Step 2, not just the total. Figure 6 and Figure 7 show, respectively, how full each boat runs and how many people are waiting system-wide as the day unfolds; both track the two commute peaks of the demand model closely, which is the expected behavior of a policy that reacts to demand rather than anticipating it. Figure 8 gives a static sense of the fleet in motion; the underlying tool can also produce an animated, minute-by-minute version and a version that can be stepped through one decision at a time, both used during development to visually confirm the fleet-assignment fix of Section 6.2,

but neither reproduces meaningfully on a printed page.

6.4 Known limitations of H0

Three limitations of this policy are worth stating explicitly rather than leaving implicit. First, the strict local-before-remote priority described above means a lightly connected pair can in principle wait indefinitely while a busier pair keeps the fleet occupied; this did not occur in the three runs reported here, but nothing in the rule itself prevents it, and a busier or more imbalanced instance could expose it. Second, fleet assignment (Section 6.2) prevents two boats from claiming the same destination in the same step, but it still assigns boats one at a time, in a fixed order, rather than searching over every possible joint assignment for the one that would serve the most people; it removes the worst failure mode without claiming to be optimal. Third, repositioning is myopic by construction, evaluating only the immediate move rather than the pickup and delivery as a planned pair, so a boat can occasionally reposition toward demand that a different, better-timed choice would have served faster. The first and third of these limitations are exactly what the next three heuristics try to reduce, and the results at the end of this section show whether each attempt actually helps.

6.5 A second coordination gap: claims that outlive a single step

Fleet assignment (Section 6.2) only looks at one simulation step at a time. It stops two boats from picking the same destination in the very same step, because it can see both decisions being made together and can discount the queue in between. It cannot do anything about a claim that stretches across several steps, and repositioning trips almost always do: a boat ordered to reposition can take many two-minute steps to arrive, and during all of that travel time the queue it is heading toward looks, to any other boat that becomes free in a later step, exactly like a queue nobody has claimed yet.

This was confirmed directly, not just argued for, with a small controlled case, run and checked step by step in the notebook that documents each heuristic’s methodology (Appendix A): at minute 100, two boats are free, one at Bryggen and one at Sandviken, and neither has any local demand. A single remote queue, Laksevåg to Kleppstø, has five people who have each been waiting 12 minutes. Both boats independently apply H0’s rule to the same snapshot and both decide the same thing: go to Laksevåg. The five people are, of course, only enough to fill one boat; the second boat makes the same trip for nothing. This is not the same bug that fleet assignment already fixed (Section 6.2), because fleet assignment discounts the queue attached to the destination a boat is ordered toward, not the specific remote queue the boat is actually travelling to serve. Two boats repositioning toward the same node for the same reason slip past that check entirely, since from fleet assignment’s point of view they are each just “going somewhere”, not “claiming this particular queue”.

6.6 H1: adding a reservation that survives the trip

H1 keeps H0’s decision rule exactly as it is: local demand first, and only when there is none does a boat reposition toward the queue that has waited longest, with ties broken by the shorter travel time. Nothing about how a single boat decides changes. What H1 adds is a small dictionary, kept outside the policy by the control loop that runs it, that records which boat is currently committed to which queue:

$$\text{reservas} : \text{boat id} \rightarrow (\text{origin}, \text{destination}). \quad (7)$$

A reservation is created only when a boat repositions, never when it serves local demand, because a local pickup happens immediately and leaves nothing pending to protect. As a concrete example: at minute 100 a boat B1, free at Bryggen with no local demand, decides

to reposition to Laksevåg because three people are waiting there for a trip to Kleppestø. This creates the entry `reservas[B1] = (laksevåg, kleppestø)`. While B1 is still travelling, every other boat sees the effective size of that queue reduced by the boat’s own capacity, not by however many people happen to be in it: with a capacity of 30, a queue of fewer than 30 people that already has a reservation against it looks, to any other free boat, like it has nobody waiting at all. New arrivals to that same queue while B1 is en route are still covered, since the discount is by capacity, not by a fixed headcount taken at the moment the reservation was made. A reservation is released the moment its boat becomes free again, since a boat that is already travelling can never be redirected mid-trip and there is no separate cancellation rule to account for; this is the only event that can end a reservation, checked automatically at the start of every decision.

The reservation only matters when several boats are free in the same step, so H1 also changes how that case is resolved. H0 processes free boats in whatever order they happen to appear in a list, which is exactly what produces the failure case above. H1 instead lets every free boat propose its best option under H0’s own rule, and commits the single proposal with the longest wait among all of them first, discounts the queue it draws from, and repeats for the remaining boats. Re-running the exact scenario above through H1 gives `{B1 : laksevåg, B2 : None}`: one boat goes, the other correctly waits, and the reservation created for B1 is what keeps a third free boat from repeating the mistake several steps later, while B1 is still travelling. This persistence was checked directly, not assumed: the same reservation dictionary was fed into two separate calls to the policy several steps apart, once right after it was created and once again after four more people had joined the same queue, and it came back byte for byte identical both times, confirming that nothing inside the policy quietly forgets or rewrites a commitment it did not just make.

6.7 H2: a cost function, but only where H0 already looks

H2 keeps H1’s reservation mechanism and its urgency-based ordering between boats unchanged. What it adds is a way to choose more carefully among several remote candidates, instead of only comparing them by whoever has waited longest with travel time as a tie-breaker. For a boat b considering a queue q , define

$$C(b, q) = T_{\text{pickup}}(b, q) - W_{\text{max}}(q), \quad (8)$$

where $T_{\text{pickup}}(b, q)$ is how many minutes it would take boat b to reach the origin of queue q (zero if it is already there), and $W_{\text{max}}(q)$ is how long the oldest person in that queue has already waited. A lower C is better: it rewards a queue that has waited long and penalizes one that is far away, in the same currency, minutes.

H2 still keeps H0’s original split between local and remote demand. For any local candidate, travel time is zero, so $C = -W_{\text{max}}(q)$, and choosing the lowest C among local candidates is exactly the same as choosing the longest wait, which is precisely H0’s rule; the cost function changes nothing there; the code for this branch is, in fact, reused from H1 unchanged. The cost function only changes anything inside the remote branch, once there is no local demand left to serve: instead of always chasing whoever has waited longest, a queue that is a little less urgent but much closer can now be preferred over one that is more urgent but far away, a genuine trade-off that H0 and H1 cannot make at all. What the cost function never does, at this stage, is let a remote queue outrank a local one. This was checked directly with a queue of five people waiting three minutes at the boat’s own node against a queue of five people waiting remotely, sweeping the remote wait up to 200 minutes: the local queue is chosen every single time, no matter how long the remote one has been waiting, because the two are still never compared against each other.

6.8 H3: one cost, no separate tiers

H3 uses the exact same cost function as H2. The one thing it removes is the split between local and remote demand that every earlier rule, including H2, still respects. Instead of first checking whether anything local exists and only considering remote queues if it does not, H3 treats every queue in the system, local and remote alike, as a candidate, and searches directly for the single cheapest boat and queue pair among all of them, committing that one, discounting it, and repeating for whichever boats are still free. A local candidate still has $T_{\text{pickup}} = 0$ and the same cost as before, but here it sits on equal footing against every remote candidate instead of being shielded from comparison. This is what lets a remote queue win outright when it is urgent enough. Substituting into the cost function gives the exact condition under which that happens, for a boat with a local wait of W_{local} facing a remote alternative with pickup time T_{pickup} and wait W_{remote} :

$$W_{\text{remote}} > T_{\text{pickup}} + W_{\text{local}}. \quad (9)$$

In words, the remote queue only wins once its extra wait is larger than the whole cost of going to fetch it, never for a small or accidental difference. This threshold was verified to the minute, not just derived on paper: with a Bryggen to Kleppestø pickup of 11.5 minutes and a local wait of 3 minutes, the formula above predicts the switch at 14.5 minutes of remote wait, and stepping the remote wait up one minute at a time shows H3 staying with the local queue at 14 minutes and switching to the remote one at 15, exactly where the formula says it should. This is the one heuristic among the four that lets a demand far from the boat outcompete a demand right next to it, which is the specific weakness of H0 stated in Section 6.4: a boat that always finishes whatever is local first, no matter how small, can in principle leave a far more urgent case waiting indefinitely elsewhere. H3 is built to close exactly that gap.

6.9 How the four rules compare

Table 7 lays out what changes and what stays the same across the four rules. None of them adds a weight that would need calibrating, none of them anticipates demand that has not arrived yet, and none of them allows a single trip to serve more than one destination; the only things that ever change are how a boat picks among candidates and how several free boats are kept from stepping on each other.

Table 7: The four dispatch heuristics, side by side.

	H0	H1	H2	H3
Per-boat rule	local, then re-	same as H0	same split, cost inside	one global cost, no split
Coordinating free boats	mote list order	by urgency	each tier by urgency	by lowest cost, globally
Reservation across steps	no	yes	yes	yes
Cost function	none	none	$C = T_{\text{pickup}} - W_{\text{max}}$, remote only	$C = T_{\text{pickup}} - W_{\text{max}}$, all queues
Can remote beat local?	never	never	never	when $W_{\text{remote}} > T_{\text{pickup}} + W_{\text{local}}$

Section 7 compares all four directly against each other, across seven fleet sizes and the same five evaluation seeds, and is where it becomes clear which of these three changes over H0 actually pays off in practice.

7 Fleet-Size Sensitivity

The four heuristics of Section 6 were run again on a larger, full-day instance (the whole operating window, 06:00 to 24:00, at the demand model’s 10% daily participation rate, Section 4), this time sweeping the fleet size itself: 4, 6, 8, 10, 12, 14, and 16 boats, each policy run on the same five evaluation seeds used throughout this report. The point of this sweep is not to verify the heuristics again (that was already done in Section 6.3), but to ask a different question: once a fleet coordination rule works correctly, does making it smarter (H1, then H2, then H3) actually translate into a better service as the fleet grows, or does the benefit fade, or even reverse?

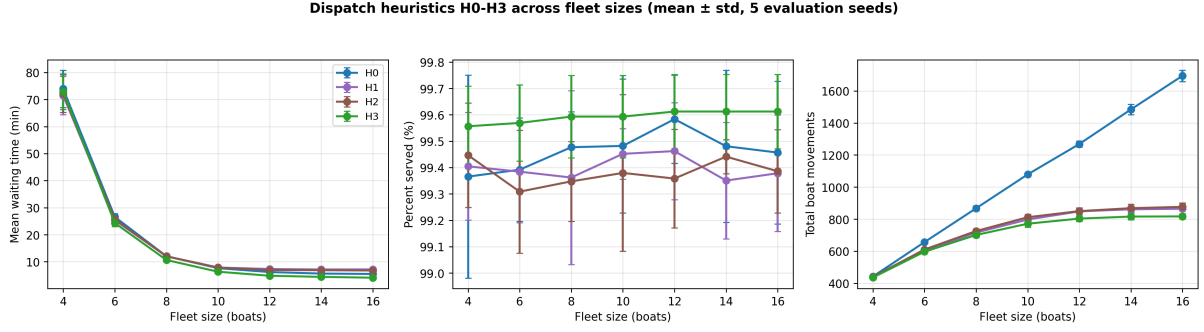


Figure 9: H0 to H3 across seven fleet sizes: mean waiting time, percent served, and total boat movements (mean $\pm$ one standard deviation over the five evaluation seeds).

Table 8: Mean waiting time (minutes) by fleet size, all four heuristics. The percentage in parenthesis is the change relative to H0 at the same fleet size; a negative number is an improvement.

Boats	H0	H1	H2	H3
4	73.9	71.5 (−3%)	72.2 (−2%)	72.9 (−1%)
6	26.6	25.4 (−4%)	25.7 (−3%)	24.3 (−8%)
8	12.0	12.0 (0%)	12.0 (0%)	10.6 (−12%)
10	7.6	7.8 (+3%)	7.8 (+3%)	6.3 (−17%)
12	6.2	7.3 (+18%)	7.0 (+13%)	4.8 (−22%)
14	5.6	7.1 (+27%)	6.8 (+22%)	4.4 (−22%)
16	5.5	7.2 (+31%)	6.7 (+23%)	4.1 (−25%)

The result is not what a reader might guess from Sections 6.6 and 6.7 alone. H1 and H2 do not beat H0 in any consistent way, and the gap against them gets worse, not better, as boats are added: H1 ends up 31% worse than H0 at 16 boats, H2 23% worse. The reason is visible in how each one spends its boats: fixing the coordination bug (H1) and adding a cost function that only ever refines the remote branch (H2) both make a boat somewhat more careful about when it is worth repositioning at all, and with more boats in the fleet, that caution more often means a boat sits idle rather than reposition toward demand that H0’s blunter, more eager rule would have already gone after. Neither H1 nor H2 has any way to weigh that caution against the cost of leaving a boat idle, so as the fleet grows and idle boats become more available, the caution costs more than it saves.

H3 is the only one of the three that beats H0 at every single fleet size tested, and the advantage grows rather than shrinks as boats are added, reaching −25% at 16 boats, while also using fewer total movements than H0, H1, or H2 at every size. This matches the design difference described in Section 6.8: H3 is the only rule that lets a remote queue outcompete a local one, so it is the only one that can use an added boat to go after whichever demand is

genuinely most urgent system wide, not just whatever happens to be closest. The improvement does flatten out: from 12 to 14 boats mean waiting time drops by 8.8%, but from 14 to 16 boats only by 6.9%, a clear sign of diminishing returns. H0 is flattening too, and faster (only -2.3% from 14 to 16 boats), which means H3 is still extracting value from additional boats at fleet sizes where H0 has essentially stopped improving.

The practical conclusion carried forward into the rest of this report is that H3 is the strongest heuristic available, at every fleet size tested, and it is the one used as the point of comparison for the reinforcement-learning agent in Section 9, alongside H0.

8 Reinforcement Learning Agent

The reference policy of Section 6 is a fixed rule with no parameters to tune; it cannot improve with experience, and it does not attempt to plan more than one move ahead. This section describes replacing it with a policy that learns: the learning method and why it fits this problem (Section 8.2), how the environment is observed and acted on (Section 8.2), the reward the policy is trained to optimize and the deliberate sequence of experiments that shaped its exact form (Sections 8.3-8.4), and the training run itself (Section 8.7), with results given together with the reference policy’s in Section 9.

8.1 Method

The learning method used is proximal policy optimization, or PPO [Schulman et al., 2017], an on-policy, actor-critic reinforcement-learning algorithm. In broad terms, PPO alternates between two phases: it runs the current policy in the environment to collect a batch of experience (states visited, actions taken, rewards received), and it then updates the policy’s parameters to make the actions that led to above-average outcomes more likely and the actions that led to below-average outcomes less likely, using that same batch. What distinguishes PPO from earlier policy-gradient methods is a clipped objective function that keeps each update close to the policy that generated the data, which in practice makes training considerably more stable: an update that would change the policy too abruptly in a single step is capped rather than applied in full. This matters here because the environment is stochastic (a fresh demand realization every episode, Section 8.6) and the reward is comparatively sparse and delayed relative to the two-minute decision interval, both conditions under which more aggressive update rules tend to destabilize training.

PPO was chosen over value-based alternatives such as deep Q-networks specifically because of the action space described in Section 5.2: one of five discrete choices per boat, which for a fleet of n_b boats is a `MultiDiscrete` space of size 5^{n_b} , not a single flat discrete choice. The standard deep Q-network implementation in the software library used for training (Section 8.2) does not support `MultiDiscrete` action spaces natively, while PPO’s policy network naturally outputs one independent categorical distribution per boat and requires no adaptation to handle several simultaneous discrete decisions.

8.2 Implementation, observation, and action

The agent is implemented with Stable-Baselines3 [Raffin et al., 2021], a maintained, widely used reinforcement-learning library built on PyTorch, driving the same Gymnasium-standard environment [Brockman et al., 2016] described in Section 5; no separate implementation of the environment exists for training, since the whole point of standardizing the environment’s interface in the first place was to let a learning library drive it without modification. The policy is Stable-Baselines3’s default `MlpPolicy`, a small feedforward network with two hidden layers of 64 units each and hyperbolic-tangent activations, shared as a common trunk between the policy head (which outputs the five-way action distribution for each boat independently) and

the value head (which estimates expected future reward from the current state, used internally by PPO to judge whether a batch of experience was better or worse than expected).

The network’s input is exactly the fixed-size numeric vector described in Section 5.1: for each boat, a one-hot encoding of its origin node, a one-hot encoding of its destination node, a normalized time-to-arrival, and a normalized occupancy (ten entries per boat, with four nodes); for each of the twelve ordered pair queues, a normalized count of people waiting and a normalized oldest-waiting-time (twenty-four entries); and two plain normalized scalars for time, minute of day and day of week (two entries). For the two-boat, four-node instance used for training (Step 1 of Section 6.3), this totals 46 entries. The network never sees the rich, dictionary-shaped state used for logging and inspection, and it never sees anything beyond this vector: no privileged information about, for instance, which specific person is waiting where, only the aggregated counts and times that the vector encodes.

8.3 Reward function

PPO trains a policy to maximize expected cumulative reward. The reward defined below is built as a cost: it is zero or negative at every step, so maximizing it is the same thing as minimizing that cost. We state it as a maximization only because that is the convention PPO and the rest of the RL literature use, not because the underlying goal is to maximize anything. What we actually want is to keep people from spending a long time in the system and to avoid moving boats without reason. That trade-off is written as one number because PPO, like essentially every policy-gradient method, needs a single scalar to optimize; it cannot be handed two separate objectives. Turning several objectives into one weighted sum is a standard step in this kind of optimization problem, usually called scalarization, and the two paragraphs after the formula explain how the weight was chosen here, not just declare it.

All the results in this report use `unidad.demanda = "personas"` (Section 4), so every unit the simulator tracks is one individual person; the formula below is written directly in terms of persons rather than the more general "person or group" wording used elsewhere in the code, since group size never actually appears in anything trained or evaluated here.

Let t index the 2-minute simulation steps, $\tau_{\text{tol}} = 12$ minutes the comfort tolerance (the longest direct navigable trip, Table 1), and $\mathcal{A}_t$ the set of people active at step t , meaning waiting in a queue or already aboard a boat. For each person $i \in \mathcal{A}_t$, let $s_{i,t}$ be the time they have spent in the system as of step t , and define their surplus as the part of that time past the tolerance:

$$\text{surplus}_{i,t} = \max(0, s_{i,t} - \tau_{\text{tol}}). \quad (10)$$

Their discomfort penalty is a quadratic function of that surplus, normalized by a fixed width $\sigma = 18$ minutes so that the result is a plain number, not a quantity in minutes:

$$\text{penalty}_{i,t} = \left(\frac{\text{surplus}_{i,t}}{\sigma} \right)^2. \quad (11)$$

Dividing by σ before squaring is what makes $\text{penalty}_{i,t}$ dimensionless: minutes over minutes cancel, leaving a ratio (how many "widths" of surplus this person has accumulated), and squaring a dimensionless ratio is still dimensionless. The square itself is deliberate: someone a minute past the tolerance barely registers, someone thirty minutes past it registers a lot more than thirty times as much. The penalty is meant to grow faster than the wait itself, so the optimizer feels increasing pressure to deal with the worst cases, not just the average one. Equation 11 has no cap: this is the version actually used to train and evaluate the agent reported in the rest of this document. A capped variant, $\min(C, \cdot)$, was tested and rejected; Section 8.4 shows why and reports that experiment.

Let m_t be the number of boats under way at step t . The full reward is

$$r_t = - \left[\sum_{i \in \mathcal{A}_t} \text{penalty}_{i,t} + w_{\text{move}} \cdot m_t \right], \quad (12)$$

with w_{move} read from configuration, not hard-coded. m_t , like the penalty, is a plain count with no unit; w_{move} is what gives it a cost in the same currency as discomfort. Table 9 shows how fast the discomfort term grows.

Table 9: Discomfort penalty by surplus, one person, uncapped (Equation 11).

Situation	Penalty
Waiting 12 minutes or less (within tolerance)	0
18 minutes total (6 minutes of surplus)	$(6/18)^2 \approx 0.11$
24 minutes total (12 minutes of surplus)	$(12/18)^2 \approx 0.44$
30 minutes total (18 minutes of surplus)	$(18/18)^2 = 1.00$
60 minutes total (48 minutes of surplus)	$(48/18)^2 \approx 7.11$

Picking w_{move} : matching scales, not guessing. Table 9 gives the scale to match: a person who has just gone one minute past the tolerance already costs about 0.003 per step. If w_{move} were, say, 0.1 (the value used in production, where this reward is never actually optimized: Section 6.3 runs the reference policy on a different, simpler formula that has no bearing on training), a single boat trip would cost more than thirty people who are each barely over tolerance, and the agent would have every incentive to leave people waiting rather than move a boat. So the search for w_{move} started from that ≈ 0.003 figure and tried values at and below it, which is what Section 8.4 reports.

This is also why the reward used for training and evaluation is a different formula from the one used to report the reference-policy runs in Section 6.3: this reward exists only to give PPO something to optimize, and its shape only matters in that context. Applying it to the reference policy would not change how that policy behaves (it is a fixed rule that never looks at the reward), and applying the production formula to the agent would defeat the point of the experiments below.

8.4 Reward-shaping experiments

Two things about the reward were undecided before spending the long training run of Section 8.7: whether capping the discomfort penalty helps or hurts a learning agent, and how large w_{move} can be before the agent starts avoiding boat trips. Both were tested directly rather than assumed. Each configuration below trained for 30,000 timesteps (a few minutes on Step 1) and was then evaluated against the reference policy on the same five held-out seeds used in Section 9.

Capped vs. uncapped discomfort (A vs. B). Configuration A uses the capped penalty,

$$\text{penalty}_{i,t} = \min \left(C, \left(\frac{\text{surplus}_{i,t}}{\sigma} \right)^2 \right), \quad C = 1.0, \quad (13)$$

with $w_{\text{move}} = 0$. Configuration B removes the cap ($C \rightarrow \infty$, i.e. Equation 11 with no modification) and is otherwise identical. The reasoning behind testing this: with the cap active, someone who has waited 35 minutes and someone who has waited 50 minutes are both charged the same maximal penalty, so the reward stops distinguishing between them past that point: an agent chasing reward has no more incentive to help the person at 50 minutes than the one

at 35. Removing the cap keeps that distinction alive for as long as someone is unserved, so the hypothesis was that B should give the agent a standing incentive to go after whoever has been waiting longest, not treat everyone past the cap as equally urgent.

Adding a small movement penalty (C1 vs. C2). Starting from the better of A and B, this pair adds w_{move} back in, at the two candidate values discussed above: C1 sets $w_{\text{move}} = 0.003$, C2 sets $w_{\text{move}} = 0.001$, to see whether a smaller weight still shapes movement without costing service.

Table 10: Reward-shaping experiments: configuration and outcome after 30,000 training timesteps, evaluated on five held-out seeds.

Configuration	Cap C	w_{move}	Percent served	Mean wait (min)	Movements
A	1.0	0	47.8%	26.2	135
B	∞	0	51.9%	21.9	125
C1	∞	0.003	52.0%	23.8	127
C2	∞	0.001	44.7%	22.5	95

Removing the cap (B) beats the capped configuration (A) on every column, including movements, so B is the base carried into C1/C2. Between those two, C1 tracks B closely on percent served while moving about as much; C2, with a smaller weight, both serves fewer people and moves less, and the fewer movements it does make are not used any better than C1’s. *Percent served is read here only as a rough dial for comparing configurations against each other within this short-training sequence, not as the metric that matters: Section 9 explains why and reports mean and maximum time in system instead once the real training budget is spent.* Neither run here gets close to the reference policy, but 30,000 timesteps is a fifth of what Section 8.7 uses for the agent actually evaluated later, so that is expected: the point of this sequence is choosing a reward shape cheaply, not producing a competitive agent yet. The combination carried forward is the uncapped penalty with $w_{\text{move}} = 0.003$ (C1).

8.5 A fuller search: eight reward formulas at full training budget

The A/B/C1/C2 sequence above is a cheap probe, thirty thousand timesteps per configuration, meant to rule out obviously bad shapes before committing to a longer run. A separate, later round of experiments repeats the same idea at the full training budget used for the agent in the rest of this report, one hundred fifty thousand timesteps per configuration, and widens the search to eight reward formulas built on genuinely different principles rather than four variations of one formula. Every configuration below is trained on the same instance (Step 1, two boats, the morning window), the same seed, and the same PPO settings as Section 8.7; only the reward changes.

- **A, pure linear time.** Charges every active person, at every step, exactly the two minutes of that step, with no term for movement. This follows directly from an identity behind Little’s Law: summing, over every step of an episode, how many people are active times the length of that step gives exactly the same number as summing, over every person, how long each one individually spent in the system. Maximizing A is therefore, by this identity, the same thing as directly minimizing total time in system, with no tolerance and no curve standing in the way.
- **B.** The same formula already used for the rest of this report (Section 8.3), the winner of the A/B/C1/C2 sequence above.

- **C, B plus potential-based shaping.** Adds a second term to B, built from a result by Ng et al. [1999]: a reward bonus of the form $\gamma\Phi(s_{t+1}) - \Phi(s_t)$, for any function Φ of the state alone, changes the reward the agent sees at every step but never changes which policy is optimal. Here $\Phi(s) = -\eta \cdot N(s)$, with $N(s)$ the number of people active in state s , so the bonus rewards making that number go down sooner, without changing the underlying objective. The intended benefit is a denser, more immediate training signal, not a different final target: B’s discomfort penalty only grows once someone has waited past the twelve minute tolerance, so a lot of early, helpful behavior gets no direct credit under B alone.
- **D.** The production reward formula, the values already used for reporting (never for training) throughout Section 6, trained here with reinforcement learning for the first time. In the version actually trained and reported here, it is the same uncapped quadratic penalty already defined in Section 8.3, with no movement term.
- **E, pure linear time, no per step tolerance.** Charges each active person the full time they have been in the system so far, added up fresh at every step, rather than the two minute increment A charges. No tolerance, no normalizer, no cap, no movement term.
- **F, pure quadratic time, not normalized.** The same squared penalty as D, but without dividing by the eighteen minute width σ first, so the number grows unnormalized. This isolates whether normalizing before squaring, on its own, matters once PPO’s own observation and reward normalization is already active.
- **G, the time added this step.** Rather than a person’s total time in system (E) or a fixed charge per step (A), G charges exactly the time that was added to the system total during this one step, computed as the change, from one step to the next, in the sum of every active person’s elapsed time, with a correction added back in for anyone who was delivered during the step (since a delivered person leaves the active count and would otherwise vanish from that difference without ever being charged for their last few minutes). This was checked to be exact, not just approximately right, by summing it over a whole ninety step episode and comparing the total against the true total time in system of every person involved: the two matched to nine decimal places.
- **H, a weighted mix of time and movement.** Combines F’s unnormalized squared time term with a count of boats currently under way, weighted 95% time and 5% movement.

Table 11: Eight reward formulas plus the reference policy, 150,000 timesteps each, evaluated on the five held-out seeds. Best value per column in bold; percent served is context only, not a ranking criterion (Section 9).

Reward	Mean sys. time (min)	Max sys. time (min)	Mean wait (min)	Movements	Served
A	37.17	122.07	22.45	164	78.1%
B	29.02	60.77	17.06	191	88.0%
C	29.25	74.07	20.21	193	84.5%
D	25.73	51.92	16.21	174	84.5%
E	41.06	123.13	24.41	172	74.8%
F	33.05	81.13	22.78	183	75.9%
G	33.46	109.74	20.72	169	74.4%
H	37.15	103.74	24.94	194	81.4%
Reference (H0)	26.05	69.74	16.42	155	88.0%

D comes out ahead of every other formula, including B, on every column that matters (mean and maximum time in system, mean wait), and it is the closest any trained agent gets to the reference policy on this instance, close enough on mean time in system to actually edge past it

(25.73 against 26.05 minutes). That D wins at all is itself worth noting: it is the raw production formula, never tuned for reinforcement learning, and the earlier, capped configuration A in the first probe above performed poorly for a related reason (a penalty that stops growing past a point gives the agent no reason to prioritize the worst case). D is uncapped and grows quickly, and here that turns out to help rather than hurt, once PPO’s own normalization is active. This is the formula carried forward into the long training run of Section 8.7 and, later, into the full-scale training of Section 8.8.

Among the four new formulas, E and F confirm that squaring the time penalty (F) is better than charging it linearly (E) on every column, which is consistent with the same design choice already made for D and B in Section 8.3: a penalty that grows faster than the wait itself pushes harder on the worst cases. G, which charges only the time added at each step rather than a person’s whole accumulated time, gets close to D’s mean and its lowest fifty percent of cases, but its worst case is much longer, suggesting a long tail that G’s flatter, per step charge does not push against as hard as D’s does. H, the weighted mix of time and movement, behaves almost identically to F rather than splitting the difference between time and movement as its 95/5 weighting might suggest; the reason turns out to be a scale problem, not a weighting problem, described next.

A scale check on H’s weighting. Running the reference policy for one full episode and adding up H’s two components separately shows why H behaves like F alone: the unweighted time term totals about 229,567 over the episode, the unweighted movement term totals 96, a ratio of roughly 2,400 to 1. Applying the nominal 95%/5% weights to numbers that far apart does not produce a 95/5 split of what the agent actually feels: the time term ends up carrying 99.998% of the weighted total, the movement term the remaining 0.002%. A weighted sum of two terms only reflects its stated weights if both terms already live on comparable scales before the weights are applied; here they did not, by roughly three orders of magnitude, and the result is a formula that reads as balanced on paper but is not in practice. This is stated here as a general point about combining several objectives into one number (exactly what Section 8.3 does for discomfort and movement, at a much smaller and previously checked ratio), not only as a comment on H.

A correctness finding, caught before it could spread further. While implementing G, a check that should have summed to exactly zero over a full episode did not, which is what first exposed a bug worth describing here because of what it shows about working with mutable state in Python. Computing G at each step needs to know the system’s total active time immediately before the step, so the first implementation simply kept a reference to the previous step’s full state object. In this codebase, that state object is built by handing it the same list and dictionary the simulator itself keeps and keeps changing, not a copy of them; a variable that appears to store “the state as it was a moment ago” was, underneath, still pointing at the live, currently-changing data, so reading it later returned the current state, not the one from before. This was confirmed directly: a state saved at the very start of an episode, re-read eight steps later, reported people already active in a system that had started completely empty. The fix does not touch the simulator, only how G (and C, which relies on the same kind of “value from one step ago”) is computed: instead of keeping a reference to the whole state object, a single number is computed and stored the moment the state is still fresh, and a number, unlike a reference to a mutable object, cannot later change out from under the code holding it. After the fix, G’s zero-sum check holds to nine decimal places over a full episode. C had already been trained under the earlier version of the code by the time this was found; Table 11 reports that run as it was, since retraining it was outside the scope of what was being checked at the time, but the potential-based shaping guarantee behind C (Ng et al.’s result, cited above) only holds for the shaping term as originally defined, so the guarantee should not be assumed to hold

exactly for that particular run.

8.6 Fresh demand on every reset

Every run reported in Section 6.3 uses one fixed table of demand, generated once and reused for the whole run, which is the right choice for verifying that the environment and the reference policy behave correctly: the policy has no parameters to overfit, so there is nothing to gain from varying the demand across runs. Training a learning agent on a single fixed table of demand is a different matter and a real risk: with enough experience against exactly the same arrivals, at exactly the same simulated minutes, a large enough network can begin to memorize that particular realization rather than learn a policy that generalizes to the underlying arrival pattern it is drawn from. To avoid this, the training environment regenerates its demand table on every reset, drawing a new realization of the same gravity-and-Poisson process of Section 4 each time, so that every training episode is an independent Monte Carlo sample of the same demand pattern rather than a repeat of the previous one. A reset called with an explicit seed still reproduces that exact realization, which is what makes the fixed evaluation seeds of Section 9 comparable across runs.

8.7 Training run

The reward-shaping choice of Section 8.4 (uncapped discomfort, $w_{\text{move}} = 0.003$) is carried into a single, longer training run of 150,000 timesteps on the Step 1 instance, roughly five times the budget used for any individual configuration in that exploratory sequence and the only long training run performed in this project; every other configuration is deliberately short, so that this one run is the point at which training budget itself, rather than reward shape, is the variable being spent.

A handful of further training settings are fixed identically across every run reported in this section and the exploratory sequence before it: the network observes and is rewarded on normalized quantities (via running mean and variance statistics computed online, frozen at their trained values for evaluation, Section 9) rather than on raw values, an entropy coefficient of 0.01 (above Stable-Baselines3’s default of zero) to keep exploration from collapsing too early, and a learning rate of 3×10^{-4} , the standard PPO default. The rollout length, meaning how many environment steps PPO collects before each policy update, is 512, smaller than Stable-Baselines3’s own default of 2048 so that updates happen roughly every five to six 90-step episodes rather than every twenty-plus, but 512 itself does not come from a formula tied to this problem, only from an earlier round of tuning. The more direct number to defend is the episode length itself, 90 steps, so that every update sees a whole number of complete episodes rather than cutting one in half; both are compared below.

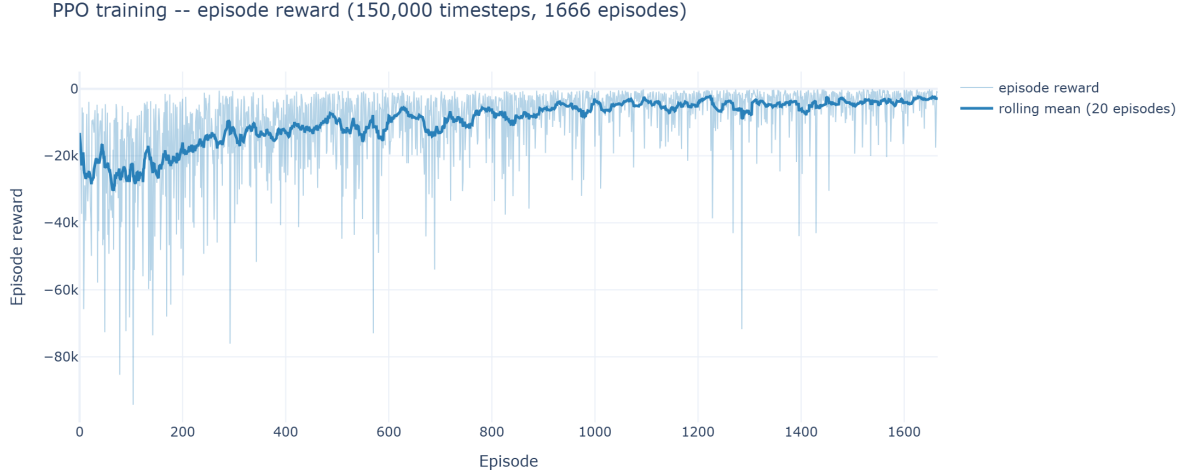


Figure 10: Episode reward over the course of the 150,000-timestep training run, $n_{\text{steps}} = 512$ (raw reward per episode and a 20-episode rolling mean).

Mean episode reward improved from $-24,871.4$ over the first 20 episodes of training to $-2,832.8$ over the last 20, almost an order of magnitude, with the final individual episodes ($-17,540.6$, $-1,905.5$, -651.3 , $-1,007.3$, -958.0) mostly settled into a low range punctuated by occasional worse episodes, consistent with the fresh, unseen demand realization drawn on every reset (Section 8.6) rather than with unstable training.

Checking the rollout length: $n_{\text{steps}} = 90$. The same 150,000-timestep run was repeated with every setting identical except n_{steps} , set to 90 instead of 512, to see whether aligning updates to episode boundaries changes the result. On the training data the two look similar: mean reward over the last 20 episodes is $-2,832.8$ for $n_{\text{steps}} = 512$ against $-2,956.2$ for $n_{\text{steps}} = 90$, close enough that the training curve alone would not obviously favor either. Evaluated on the same five held-out seeds used in Section 9, the two diverge sharply:

Table 12: $n_{\text{steps}} = 512$ vs. $n_{\text{steps}} = 90$, same reward and same training budget, evaluated on the five held-out seeds.

Metric	$n_{\text{steps}} = 512$	$n_{\text{steps}} = 90$	Reference policy
Mean time in system (min)	29.0	30.2	26.1
Maximum time in system (min)	60.8	73.7	69.7
Mean waiting time (min)	17.1	19.9	16.4
Movements (5 episodes)	191	179	155
Reward (5 episodes)	-3772.0	$-28,082.5$	-5259.0
Percent served (context only)	88.0%	77.5%	88.0%

$n_{\text{steps}} = 90$ loses on every column, including the two that matter most: its worst case (73.7 minutes) is worse than the reference policy’s, not just worse than $n_{\text{steps}} = 512$ ’s, and its evaluation reward ($-28,082.5$) is close to where $n_{\text{steps}} = 512$ started training, not where it ended. Training performance and evaluation performance pulled apart for this configuration even though both runs looked similar while training. The most likely reason: with $n_{\text{steps}} = 90$ the rollout buffer holds exactly one episode, so PPO updates about 1,666 times over the run (once per episode) instead of about 293 times, each update built from an order of magnitude less experience. More frequent updates on smaller batches make each gradient step noisier and more

tied to that one episode’s particular random demand, which can look like steady improvement during training while leaving the policy less robust to demand it has not seen. $n_{\text{steps}} = 512$, despite not being chosen for a principled reason either (Section 8.7), ends up giving each update a wider, less noisy sample to learn from, and that turns out to matter more than lining updates up with episode boundaries. The agent used throughout Section 9 is the $n_{\text{steps}} = 512$ run; $n_{\text{steps}} = 90$ is reported here as a result that did not hold up under evaluation, not adopted.

8.8 Scaling up: training at the full instance

Everything up to this point, the reward search of Sections 8.4 and 8.5 and the training run just described, uses Step 1: two boats and a three hour morning window. That instance was chosen because it is small enough to iterate on quickly, but it is not the instance the rest of this report is actually about: the fleet-size sweep of Section 7 and the heuristics it compares run on the full operating day with twelve boats. An agent trained on two boats cannot simply be pointed at twelve: the input vector described in Section 8.2 has a size that depends directly on the number of boats and pair queues, so a network trained for one fleet size is not even the right shape to load for another. A second, separate training run is therefore needed at the full scale before the agent can be compared against the heuristics on equal terms, which is what this section describes.

The setup carries over the winning choice of Section 8.5 without change: reward D, the same uncapped quadratic discomfort penalty, no movement term, and the same PPO settings (learning rate, entropy coefficient, VecNormalize) used throughout. The instance itself is the full-day, twelve-boat one used for the heuristics comparison in Section 7: 06:00 to 24:00, capacity 30 per boat, five hundred forty two-minute steps per episode. The rollout length is set to match, 540 steps, so that every PPO update sees a whole number of complete episodes, the same reasoning already applied to the smaller instance in Section 8.7. Before committing to a long run, a short automated check confirmed the basics could not have silently drifted: the episode length, the boat count and capacity, the reward configuration actually being read, and that two calls to reset with the same explicit seed return identical demand while two calls with no seed return different demand, all matched what was intended.

Two practical changes were made to this run purely to make better use of the computer it ran on, neither of which touches the reward, the hyperparameters, or the environment’s mechanics. First, the environment runs in two copies in parallel rather than one, so that the computer’s several processor cores can each advance a copy of the simulation at the same time instead of one core doing all the work alone; this cuts wall clock time roughly in half for the same number of environment steps. Second, an unrelated piece of background software on the same machine (a cloud file sync client) was found, partway through an earlier attempt, to be competing heavily for processor time with the training run itself; pausing it removed that source of slowdown. These are computational, not scientific, decisions, and they are recorded here for transparency rather than because they change what was learned. The one consequence worth stating plainly is the total training budget: an initial attempt at four parallel environments and one million timesteps ran into memory pressure on the training machine and had to be stopped partway through, so the run actually completed here uses two parallel environments and a reduced budget of seven hundred thousand timesteps, chosen to fit comfortably in the time available rather than derived from any formula.

The completed run reached 700,920 timesteps (slightly over the seven hundred thousand target, since training only stops at the end of a whole update), 649 policy updates, and about 1,892 complete episodes, in a little under four hours of wall clock time. The resulting model and its normalization statistics are what Section 10 evaluates against the heuristics of Sections 6 and 7, on the same twelve-boat, full-day instance and the same five evaluation seeds used everywhere else in this report.

9 Results: Agent vs. Reference Policy at Small Scale

This section keeps the comparison originally run for this project’s first trained agent, at the two-boat scale used to develop and check the training pipeline itself. It is a useful checkpoint precisely because the instance is small enough to read a single run by eye, but the fleet-size sweep of Section 7 already showed that a heuristic’s ranking at four boats does not always predict its ranking at twelve, so this comparison should be read as a sanity check on the method, not as the project’s main result. Section 10 repeats the same kind of comparison, agent against the strongest heuristic, on the full twelve-boat instance the rest of this report is about, and is where the main conclusion of this project should be read from.

The trained agent of Section 8.7 and the reference policy of Section 6 are run on the same five evaluation seeds, 1001 through 1005, chosen outside the range of seeds used to generate training episodes so that neither policy is evaluated on demand it could have seen before. Because both policies act on the same `EntornoDemandaAleatoria.reset(seed=...)` call (Section 8.6), a given seed produces the identical demand table for both, so any difference in outcome reflects the policy, not the demand.

Why percent served is not the headline metric. Percent served is reported below for context, but it is not what either policy is actually being judged on, and it should not be read as the main result. The environment does not lose anyone (Section 5): a person who has not boarded a boat by the time a run ends is not gone, they are simply still in a queue or still aboard when the clock stops, and would be served if the run kept going a bit longer. So “percent served” mixes two different things: how well the policy dispatches boats, and how much of the operating window happened to be left when the last few people arrived, which is arbitrary and has nothing to do with policy quality. The two things the reward (Section 8.3) actually penalizes, and the two things worth comparing here, are how long people spend in the system, mean and especially worst case, and how many times the fleet moved to achieve that.

9.1 Aggregate comparison, five seeds

Table 13: Agent vs. reference policy, aggregated over the five evaluation seeds. Reward is computed with the training reward (Section 8.4) for both policies, so that the total is comparable between them.

Metric	PPO agent	Reference policy	Difference
Mean time in system (min)	29.0	26.1	+3.0
Maximum time in system (min)	60.8	69.7	−9.0
Mean waiting time (min)	17.1	16.4	+0.6
Total movements (5 episodes)	191	155	+36
Total reward (5 episodes)	−3772.0	−5259.0	+1487.0
Percent served (context only, see above)	88.0%	88.0%	0.0 pp

The agent beats the reference policy on the worst case (60.8 vs. 69.7 minutes, the number the uncapped reward pushes on directly) and on the reward it was trained against, at the cost of a higher mean time in system and 36 more boat movements over the five episodes. That trade-off is the direct, expected consequence of removing the discomfort cap (Section 8.4): with no ceiling on any one person’s penalty, the agent keeps a standing incentive to stop any single case from growing into a very long wait, even if that means accepting a slightly worse average and moving boats more to chase down the worst cases. Percent served happens to come out identical for both, 748 of 850 people over the five seeds. That is interesting mainly because it means the

two are not being compared on wildly different operating points, not, per the paragraph above, evidence of anything about service quality on its own.

The result is not uniform across seeds. The agent’s reward is worse than the reference policy’s on three of the five (1001, 1003, 1005), and the aggregate is carried by seed 1004, where the reference policy has its single worst run of the ten (-4091.1) while the agent handles the same demand comfortably (-1525.6). That is consistent with an agent that is more consistent in its worst case rather than uniformly better episode by episode. Cross-checking the decision log directly: the agent chooses to wait in only 5.4% of its decisions (11 of 202), against 47.5% for the reference policy (140 of 295). The extra movements above are not idle waste: the agent is genuinely dispatching more actively, not just spending the movement penalty on nothing.

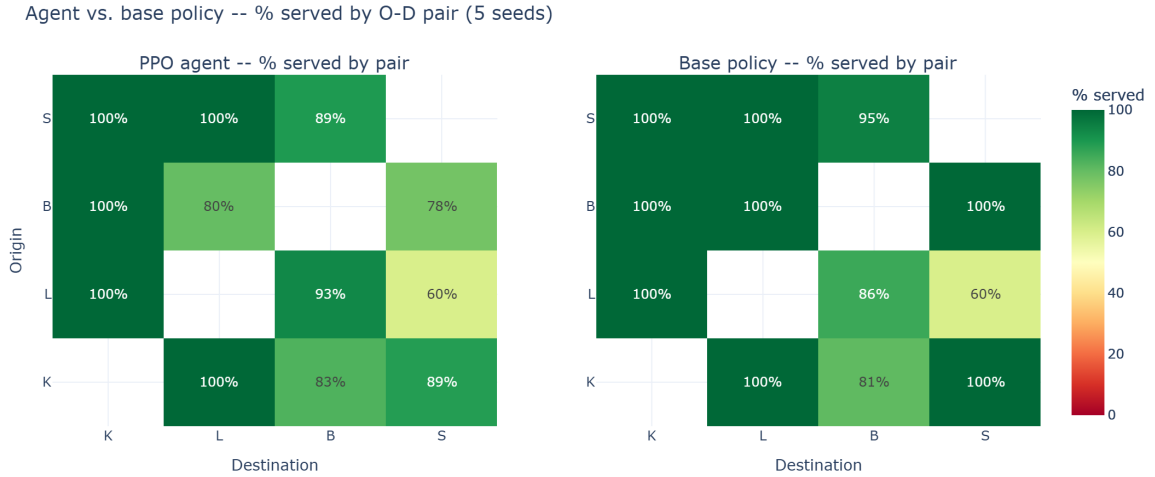


Figure 11: Percent served by origin-destination pair, agent vs. reference policy, aggregated over the five evaluation seeds.

9.2 Detailed comparison on a single seed

To see behavior, not just the aggregate, seed 1001 is examined with the same package of metrics and figures produced for the reference-policy verification of Section 6.3 (waiting profile, fleet occupancy over time, compliance heatmap by pair, reward breakdown, and backlog by pair), computed separately for the agent and for the reference policy on that one seed, so that the concrete pattern of a single run is visible side by side for both.

On this specific seed the reference policy has a clearly shorter mean wait than the agent (9.4 vs. 17.4 minutes), which is the opposite of what the aggregate reward comparison above might suggest. This is why the detailed view matters: seed 1001 is one of the three individual seeds where the reference policy’s reward is better than the agent’s, and the aggregate advantage comes from the reward being sensitive to the worst case across all five seeds, not from the agent doing better seed by seed. Both readings are correct at the same time: the agent is the stronger choice by the worst-case-aware objective it was trained on, and the reference policy can still be the better choice on a representative single run like this one. (Percent served happens to move the same direction here too, 124 of 135 for the reference policy against 110 of 135 for the agent, but per the caveat above that is not the reason to prefer either policy on this seed; the wait-time gap is.)

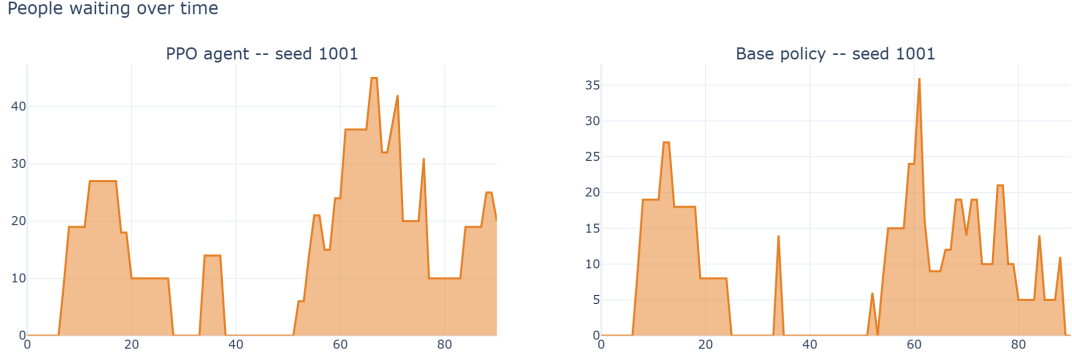


Figure 12: People waiting over time, seed 1001: agent (left) vs. reference policy (right).

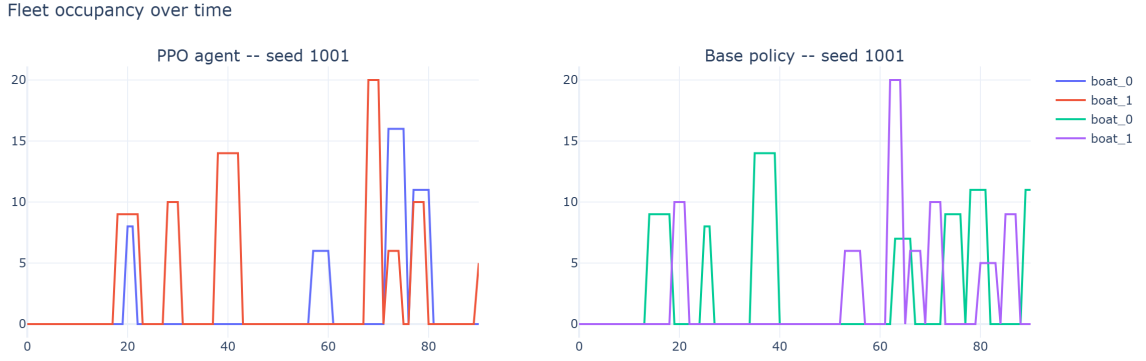


Figure 13: Fleet occupancy over time, seed 1001: agent (left) vs. reference policy (right).

10 Results: Full-Day Comparison

This is the main comparison of the project: the agent trained at full scale in Section 8.8 against H0 and H3, the weakest and the strongest of the four heuristics from Sections 6 and 7, all three run on the exact instance the rest of this report has been building toward, twelve boats, capacity 30, the full operating day from 06:00 to 24:00, and the same five held-out evaluation seeds used everywhere else. H1 and H2 are left out here because Section 7 already showed neither one beats H0 consistently at this scale, so H0 and H3 already bracket the range worth comparing the agent against. H0 and H3's numbers below are the same ones already reported in Section 7 at the twelve-boat row, not recomputed; only the agent is evaluated fresh here, on the same five seeds, using the model of Section 8.8.

As a check before trusting the comparison at all, the amount of demand generated for a given seed was confirmed to match exactly between the heuristics, which read a fixed table of demand prepared ahead of time, and the agent, which generates its own demand table on the fly from the same seed (Section 8.6): for seed 1001, both paths produce exactly 8,324 people, confirming that "the same seed" really does mean the same demand for all three.

10.1 Aggregate result, five seeds

Table 14: H0, H3, and the trained agent, twelve boats, full day, mean over the five evaluation seeds.

Metric	H0	H3	PPO agent
Percent served	99.58%	99.61%	99.39%
Backlog at close	34.2	31.8	50.2
Mean waiting time (min)	6.18	4.81	12.76
95th percentile wait (min)	15.71	12.27	37.25
Maximum wait (min)	28.03	18.36	105.40
Mean time in system (min)	15.97	14.60	22.55
Total movements	1268.6	803.2	1149.8
Empty movements	704.0	200.6	660.6
Loaded movements	564.6	602.6	489.2

The 95th percentile and maximum time in system are also available for the agent (46.17 and 115.40 minutes) but not for H0 and H3, since the fleet-size sweep of Section 7 did not record them; they are not recomputed here to avoid mixing a number just calculated with numbers taken as given, so they are reported for the agent alone, for completeness, without a heuristic to set them against.

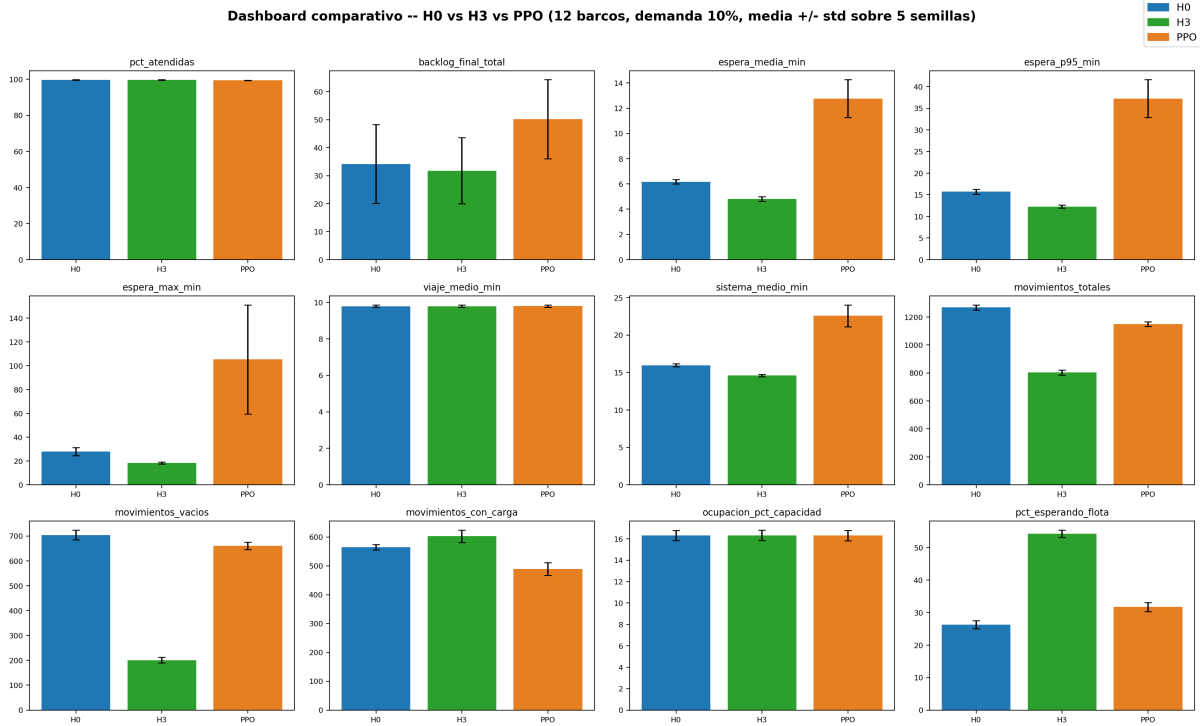


Figure 14: All headline metrics side by side, H0 vs. H3 vs. the trained agent, twelve boats, full day, mean $\pm$ one standard deviation over five seeds.

% atendidas por par O-D -- H0 vs H3 vs PPO (semilla 1001)

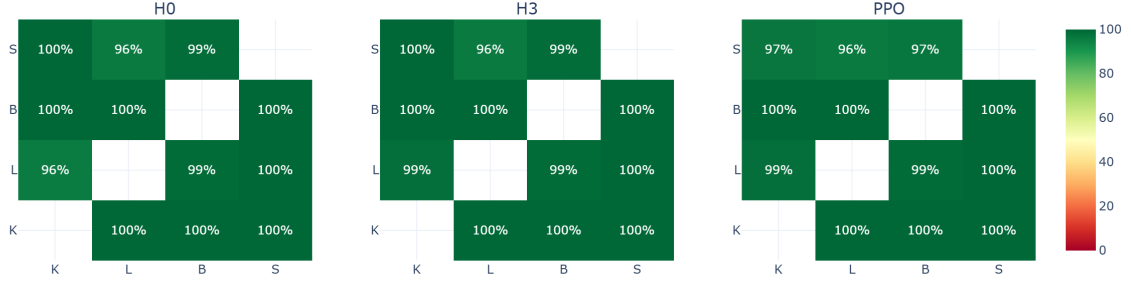


Figure 15: Percent served by origin-destination pair, H0 vs. H3 vs. the trained agent, seed 1001.

10.2 Reading the result honestly

At this scale, the trained agent does not beat either heuristic. It comes close on percent served (99.39% against 99.58% and 99.61%), but every metric this report has treated as the one that actually matters throughout (mean wait, the 95th percentile, the worst case, time in system) is clearly worse for the agent, by a factor of two to four depending on the column, and the backlog left over at the end of the day is larger too. H3 remains the strongest policy of the three by a wide margin, and it does so while moving the fleet less than either H0 or the agent, so this is not a case of H3 simply working the boats harder. This is stated plainly and without qualification because the reward the agent was trained on is not the criterion for this comparison (Section 9 already explains why, and the same reasoning applies here): the agent should not be called better because its own training signal looks favorable, only because the same operational numbers used to judge every heuristic in this report say so, and here they do not.

It is worth asking why, given that the same reward, D , was the strongest performer of eight tested at the smaller, two-boat scale (Section 8.5), and even edged past the reference policy there. Three things changed at once between that result and this one, and this report cannot cleanly separate their individual contribution. First, the action space grows sharply with the fleet: each boat still chooses among five options, but the joint choice for the whole fleet is one of 5^{12} combinations at this scale against 5^2 at the smaller one, a much larger space for the same policy network architecture (Section 8.2) to search. Second, the training budget did grow, from 150,000 to about 700,000 timesteps, but far less than the instance did: the smaller instance's episodes are 90 steps long, so 150,000 timesteps is around 1,600 full episodes of experience, while the larger instance's 540 step episodes mean 700,000 timesteps is only around 1,300 episodes, fewer complete passes through the problem despite the larger raw timestep count. Third, the twelve-boat instance was reached in a single attempt rather than through the same kind of deliberate, evidence-based search this report ran for the reward itself (Section 8.5) and for the rollout length (Section 8.7); it is entirely possible that some setting tuned for the two-boat instance, the rollout length in particular, is no longer a good choice once the episode itself is six times longer, and nothing in this report tests that directly at the larger scale. None of this excuses the result or explains it away; it points at what a next attempt at this scale would need to try first.

11 Assumptions and Limitations

This project makes a number of explicit simplifying choices, stated here together rather than left scattered or implicit.

No real demand data. The service does not exist, so no observed origin-destination counts are available anywhere in this report; Section 4’s gravity-style intensity score, its distance-decay term, its role assignment by time band, and its daily participation rate are all team decisions made in the absence of that data, anchored in real population and employment figures but not validated against ridership.

No passenger patience, by design. The environment does not remove anyone from a queue for having waited too long (Section 5); every person generated is either served or still in the system when a run ends. This was a deliberate choice to obtain uncensored waiting-time statistics (Section 6.3), but it also means neither policy has any pressure resembling a hard service-level deadline: the reference policy never consults the reward at all, and the agent is trained on the uncapped discomfort penalty (Section 8.3), which grows without bound the longer someone waits rather than plateauing.

None of the four heuristics is an optimizer. H0 through H3 (Sections 6 and 6.6-6.8) are considered rules, each checked by hand-built cases and compared honestly against the others, but none of them searches for the best possible joint assignment of boats to demand; every comparison in this report, including the agent’s, should be read as one method against another, not against a known optimum.

The reward search and the rollout-length check were only ever done at the small, two-boat scale. Sections 8.4 through 8.7 settle on a reward formula and a rollout length using the two-boat morning instance exclusively, because it is cheap enough to run many configurations on. The full-scale training run of Section 8.8 carries that same reward and rollout length forward without re-checking either one at the larger size, and Section 10 discusses this directly as one of the likely reasons the full-scale agent does not perform as well, relative to the heuristics, as the small-scale one did.

The full-scale training run was attempted once. Unlike the reward search, which compared eight formulas, or the rollout-length check, which compared two settings, the twelve-boat training of Section 8.8 was run a single time, with a training budget chosen to fit the time available rather than tuned for this instance specifically; a different budget, a different rollout length, or a repeated run with a different random seed might change the result in either direction, and this report does not have the evidence to say by how much.

Reward shape was explored, not exhaustively searched. Sections 8.4 and 8.5 together test twelve configurations along several design axes (discomfort cap, movement weight, linear vs. quadratic, potential-based shaping, a multi-objective weighting), which is a wider search than a single guess but still not every combination worth trying; the chosen configuration, reward D, is a reasonable, evidence-based choice rather than a demonstrably optimal one. One of the twelve, reward C, was later found to have been trained under a software bug that changed what its shaping term actually computed (Section 8.5); its result is reported as it was obtained, not retrained, and is flagged there rather than treated as a clean data point.

12 Conclusion

This project set out to build the simulation and dispatch logic for a schedule-free, on-demand water transport service in Bergen, from the routing and demand model that feed it (Sections 4 onward) through a family of fixed dispatch rules and, finally, a learned one, compared honestly against each other on the same instance.

The four fixed heuristics tell a clear, evidence-based story. H0, nearest-available with fleet coordination (Section 6), is more than a placeholder: verified at three increasing time scales, it serves essentially all demand and its coordination step is what lets it work at all once more than one boat shares the same demand picture. H1 fixes a second coordination gap, one that H0’s own fix did not reach, by adding a reservation that survives across simulation steps, not just within one. H2 adds a cost function but keeps H0’s local-first split. H3 removes that split entirely, letting any queue in the system compete on equal footing, and this is the one change that actually pays off: across seven fleet sizes, H3 is the only heuristic that beats H0 consistently, and its advantage grows rather than shrinks as the fleet grows (Section 7), while H1 and H2 fall further behind H0 as more boats are added. This is not a small or expected result: adding a cost function alone (H2) is not enough, and only removing the local-first restriction that costs are computed under (H3) turns that cost function into a genuine improvement.

The reinforcement-learning agent was developed with the same evidence-first approach: twelve different reward formulas were tried, at a small, fast-iterating scale, before settling on one (Section 8.5), and a rollout-length choice was checked directly rather than assumed (Section 8.7) once training performance and evaluation performance turned out to disagree for a plausible-looking default. At that same small scale, the chosen reward even edged past the reference policy on mean time in system. Trained again at the full scale this report is really about, twelve boats and a full operating day (Section 8.8), the agent does not repeat that result: H0 and, especially, H3 both serve people faster and more predictably than the trained agent at this scale, on every metric this report has treated as the one that matters, not only on the reward the agent happened to be trained on (Section 10). This is reported as it was measured, without softening it, because the point of building the heuristics first and holding them to real operational metrics throughout was precisely so that a result like this one could not be waved away.

Taken together, the clearest finding of this project is that a carefully built rule, one that removes an unnecessary restriction on what it is allowed to compare (H3), was harder to beat than expected, and a learning method that looked strong at small scale did not automatically carry that strength to the scale that actually matters. Both of those are useful things to know before committing to either approach for a real service, and both point at the same next step: whatever is tried next, whether a better-tuned agent or a further refinement of the heuristic, needs to be checked at the full scale directly, not assumed to transfer from a smaller one.

The most direct next steps follow from the limitations of Section 11: repeating the reward and rollout-length search directly at the twelve-boat scale, rather than carrying over choices made at the smaller one; training the full-scale agent for longer, or more than once, to see how much of the current gap is a training-budget effect rather than a ceiling; retraining reward C under the corrected code now that the aliasing bug behind it is understood (Section 8.5); and, longer term, using the uncensored waiting-time percentiles this environment already produces to propose a concrete service-time guarantee, the kind of commitment a real schedule-free service would need to state and hold itself to.

A Where Everything Lives in This Project

This report is one part of a larger project kept under version control (git), with every simulator, policy, training script, and result file organized into folders by what it is responsible for. This appendix is a short map of that project, for anyone who wants to look at the code or the raw results behind a number in this report, or rerun something themselves.

The overall layout. Six top-level folders each own one piece of the project, and every later piece reads from the earlier ones rather than copying anything:

- `bergen-boats/` holds the base instance: the four stops, the navigable routing module of Section 3, and the resulting travel-time matrix.
- `demand/` holds the synthetic demand model of Section 4: the population and employment data, the intensity formula, and the code that draws arrivals.
- `simulador/` is the simulation engine itself (Section 5): the environment, the state and reward definitions, the metrics used throughout this report, and the mapping and plotting code shared by every later piece.
- `politica_base/` and `heuristicas/` hold H0 and H1 through H3 respectively (Sections 6-6.8), together with the fleet-size sweep of Section 7.
- `modelo_rl/` holds the reinforcement-learning agent: the training code, the reward-formula experiments of Sections 8.4 and 8.5 (the latter under `modelo_rl/prueba_rewards/`), and the full-scale training run of Section 8.8.
- `comparacion/` holds every side-by-side comparison between policies, including both results sections of this report (Sections 9 and 10).

A shared rule. Every one of these folders keeps its own settings in a single `config/instance.yaml` file (fleet size, capacity, reward parameters, evaluation seeds, and so on) and every other folder that needs one of those settings reads that same file rather than copying the value into its own code; this is why, for instance, the twelve-boat instance used in Section 7 and the one used in Section 10 are guaranteed to be the same instance, not two numbers that happen to match.

Finding a specific result. Each folder also keeps a `README.md` file explaining, in plain language, what is in it and why, and most also keep an `outputs/` or `output/` folder with the raw CSV result tables and figures behind whatever this report summarizes. In particular, the five controlled cases used to check H1 through H3 in Section 6.6 live in `heuristicas/notebooks/01.metodologia.heuristicas.ipynb`, run live rather than only asserted; the full fleet-size sweep behind Section 7 is in `heuristicas/notebooks/03.comparacion.flota.ipynb`, with its raw numbers in `heuristicas/outputs/resultados/`; the eight-formula reward search of Section 8.5, including the scale-check and the aliasing bug described there, is written up in full in `modelo_rl/prueba_rewards/README.md`; and the full-scale training and final comparison of Sections 8.8 and 10 are in `modelo_rl/ppo.dia_10pct.12barcos_reward.D_1M/` and `comparacion/dia_completo/` respectively, the latter also holding an animated, minute-by-minute video of all three policies side by side over the morning hours, the same kind of visualization Figure 8 could only show as a handful of static frames here.

References

- Caroline Braathen, Julio C. Goez, and Mario Guajardo. Autonomous ferries in light of labor regulations – a passenger perspective. *Maritime Transport Research*, 7:100115, 2024. doi: 10.1016/j.martra.2024.100115.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01720*, 2016.
- Yong Gu and Stein W. Wallace. Operational benefits of autonomous vessels in logistics – a case of autonomous water-taxis in Bergen. *Transportation Research Part E: Logistics and Transportation Review*, 154:102456, 2021. doi: 10.1016/j.tre.2021.102456.

- Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- Gabriel E. Sánchez-Martínez, Nigel H. M. Wilson, and Haris N. Koutsopoulos. Schedule-free high-frequency transit operations. *Public Transport*, 9:285–305, 2016. doi: 10.1007/s12469-016-0129-8.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.